Martin-Luther-Universität Halle-Wittenberg

Bachelorarbeit

Entwurf und Entwicklung eines KI-gestützten Verfahrens zur Auswahl einer (Meta-)Heuristik für das Touristentour-Design-Problem

vorgelegt von: Luca Hoffmann

vorgelegt am: 18. November 2025

Betreuer: Prof. Dr. Matthias Müller-Hannemann

Inhaltsverzeichnis

1. Einleitung **1**

2. Theoretische Grundlagen **3**
2.1. Ziel des Kapitels 3
2.2. Problemdefinition 3
2.3. Verwandte Probleme 4
2.4. Relevante Konzepte der Graphentheorie 5
2.5. Heuristiken und Metaheuristiken 6
2.6. Theorie der KI zur Algorithmenauswahl 7
2.7. Bewertungs- und Vergleichsmetriken 8
2.8. Komplexität 9
2.9. Zusammenfassung 10

3. Modellentwicklung **12**
3.1. Zielsetzung und Annahmen 12
3.2. Instanzbasis und Datenaufbereitung 12
3.2.1. Quellen der Instanzen 12
3.2.2. Knotenlage und Distanzen 13
3.2.3. Instanzgrößen und Budgetniveau 13
3.2.4. Scores 13
3.2.5. Datenformat und Einlesen 13
3.2.6. Reproduzierbarkeit 14
3.3. Konstruktionsphase 14
3.3.1. Ausgangspunkt und Notation 14
3.3.2. Einfügekosten und Score-Zuwachs 14
3.3.3. Greedy Konstruktion 14
3.3.4. GRASP Konstruktion 15
3.3.5. Kriterien von Greedy und GRASP 15
3.3.6. Abbruch und Gültigkeit 15
3.4. Lokale Suche und Nachbarschaften 15
3.4.1. Reihenfolgeändernde Nachbarschaften 16
3.4.1.1. 2-opt 16
3.4.1.2. Relocate 16
3.4.1.3. Swap 16
3.4.1.4. Blockverschiebung 17
3.4.2. Mengenändernde Nachbarschaften 17
3.4.2.1. Add 17
3.4.2.2. Drop 17
3.4.2.3. Replace 17
3.4.3. Suchstrategie und Abbruch 17
3.4.4. Einordnung in die Metaheuristiken 18
3.5. Metaheuristiken 18
3.6. Parameterauswahl 19
3.6.1. Zeitbudgets 19
3.6.2. Gemeinsame Bewertung 19

3.6.3. GRASP . . . 19
3.6.4. Iterated Local Search . . . 20
3.6.5. Variable Neighborhood Search . . . 20
3.6.6. Zufall und Seeds . . . 20
3.7. Algorithmenauswahl . . . 20
3.7.1. Zielsetzung . . . 20
3.7.2. Merkmalsraum . . . 21
3.7.3. Zielvariablen . . . 21
3.7.4. Lernverfahren . . . 21
3.7.5. Auswertung . . . 21
3.8. Modellvarianten . . . 21
3.9. Verwendete Bibliotheken und Tools . . . 22
3.10. Evaluationsprotokoll . . . 22

4. Ergebnisse . . . 23
4.1. mittlerer Score vs. Zeitlimit . . . 23
4.2. Abstand zum VBS vs. Limit . . . 25
4.3. Within-δ-Raten . . . 25
4.4. Confusion je Limit . . . 27
4.5. Leistungsprofile . . . 29
4.6. Win–Tie–Loss vs SBS-Score . . . 31
4.7. ΔScore + CI . . . 32
4.8. Wilcoxon-Test . . . 33
4.9. Klassifikationsgüte . . . 35
4.10. Entscheidungsqualität und Merkmalsbedeutung . . . 36

5. Diskussion und Einordnung der Ergebnisse . . . 38
5.1. Zusammenfassung . . . 38
5.2. Kernbefunde . . . 38
5.2.1. Einordnung . . . 38
5.2.2. Fehlerbild und Treffgenauigkeit . . . 38
5.2.3. Heterogenität und Risiko . . . 38
5.3. Scorebasierter Vergleich mit dem SBS . . . 38
5.4. Konfidenzintervalle und gepaarte Tests . . . 39
5.5. Per-Limit gegenüber global . . . 39
5.6. Alternative Erklärungen . . . 39
5.7. Praktische Implikationen . . . 39
5.8. Validität und Grenzen . . . 40
5.9. Schlussfolgerung . . . 40

6. Ausblick . . . 41
6.1. Ziel und Rahmen . . . 41
6.2. Entscheidungen bei Unsicherheit . . . 41
6.3. Zweiter Versuch bei knappen Fällen . . . 41
6.4. Wahl zwischen global und per-Limit . . . 41
6.5. Robustheit auf anderen Instanzen . . . 41
6.6. Aufwand und Latenz . . . 42

6.7. Betrieb und Monitoring 42

7. Fazit **43**

Literaturverzeichnis **45**

A. Confusion-Matrizen **47**
A.1. Globales Modell 47
A.2. per-Limit Modell 48

B. Top-2 Merkmale **49**

1. Einleitung

‚Künstliche Intelligenz', ein Begriff, welcher die heutige Gesellschaft grundlegend verändert hat und fest in ihr verankert ist. Meist wird er mittlerweile mit Large-Language-Modellen (LLMs) wie *ChatGPT*, *Gemini*, *Deepseek* oder *Meta* assoziiert. Die Nützlichkeit von Machine-Learning-Modellen beschränkt sich jedoch nicht nur auf diese Klasse von Modellen, welche für alltägliche Anwendungen wie das Erstellen eines Rezepts oder das schnelle Nachschlagen von Informationen zu verschiedenen Themen. Vielmehr können solche Modelle auch bei fundamentalen Problemen der Informatik eingesetzt werden.

Spezifischer betrachtet ermöglichen Machine-Learning-Methoden neue Perspektiven auf algorithmische Fragestellungen und eröffnen dadurch neue Lösungsansätze. Ein konkretes Anwendungsfeld ist die Analyse von Algorithmen auf Graphen hinsichtlich ihrer Laufzeit und Ergebnisqualität. Dabei kommen sogenannte Metaheuristiken zum Einsatz. Diese versuchen, abhängig von strukturellen Eigenschaften des Graphen, Zusammenhänge zu erkennen und darauf basierend aus einer Menge vorab definierter Heuristiken die geeignetste auszuwählen. Solche Verfahren sind in der Literatur bereits für einige klassische Probleme gut untersucht worden. Dazu zählen das bekannte Traveling-Salesman-Problem (TSP), Varianten wie das Prize-Collecting-TSP (PCTSP), sowie das Vehicle-Routing-Problem (VRP). Studien zeigen, dass adaptive Metaheuristiken in diesen Kontexten nicht nur schnellere Laufzeiten ermöglichen, sondern dabei auch mindestens vergleichbare, teils sogar bessere Lösungen im Vergleich zu festen Heuristiken liefern können[26].

Ein bisher weniger erforschtes, aber praxisrelevantes Problemfeld ist das sogenannte *Touristentour-Design-Problem* (TTDP). Dabei geht es darum, innerhalb eines gegebenen Graphen eine Tour zu finden, bei der jeder Knoten mit einem individuellen *Score* versehen ist und die Kanten verschiedene Distanzen darstellen. Scores beschreiben die Attraktivität oder den Nutzen eines Ortes innerhalb der Tour. Zusätzlich ist ein Budget bezüglich der maximal zulässigen Distanz vorgegeben. Ziel ist es, eine Tour zu bestimmen, die innerhalb dieses Budgets möglichst viele Knoten erzielt, also die Summe der Scores von besuchten Knoten maximiert.

In dieser Arbeit wird untersucht, inwiefern sich die Methodik adaptiver Metaheuristiken auf das TTDP übertragen lässt und ob dadurch bessere oder effizientere Lösungen im Vergleich zu klassischen Heuristiken gefunden werden können. Daraus ergeben sich die folgenden zentralen Forschungsfragen: ‚Welche Graphenmerkmale eignen sich zur Klassifikation?' und ‚In welchem Maße verbessern sich Lösungsqualität und Laufzeit gegenüber klassischen Heuristiken?' Zur Beantwortung dieser Fragen werden zunächst zahlreiche synthetische Graphen erzeugt, die unterschiedliche strukturelle und realitätsnahe Eigenschaften abbilden. Auf diesen Instanzen werden klassische Heuristiken wie das Greedy Verfahren, Iterated Local Search, Variable Neighbour Search und Greedy Random Adaptive Search Procedure angewendet. Die erzielten Ergebnisse hinsichtlich Score und Laufzeit werden dokumentiert. Anschließend erfolgt das Training eines Random-Forest-Klassifikators, der anhand von strukturellen Grapheigenschaften die jeweils beste Heuristik vorhersagen soll. Zu diesen Grapheigenschaften zählen sowohl einfache Kennzahlen wie die Knotenanzahl, der durchschnittliche Knotengrad, die Dichte oder die Varianz der Kantengewichte, als auch komplexere Eigenschaften wie der Clusteringkoeffizient, die Anzahl der Zusammenhangskomponenten oder verschiedene Zentralitätsmaße. Die Auswahl dieser Merkmale ist entscheidend für die Lernleistung des Klassifikators.

Die Entscheidung für den Einsatz eines Random Forest als Modell zur Klassifikation basiert auf mehreren Gründen: Er liefert robuste Ergebnisse, ist vergleichsweise resistent gegenüber Overfitting und bietet durch die Feature-Importance-Metrik eine nachvollziehbare Interpretierbarkeit[1, 3]. Das ist ein Maß dafür, wie stark ein bestimmtes Eingangsmerkmal die Entscheidung eines Modells beeinflusst hat. Zudem ist er besonders gut für große Datensätze geeignet, wie sie in dieser Arbeit verwendet werden. Die Bewertung des Gesamtansatzes erfolgt durch eine umfassende Benchmark-Analyse, bei der sowohl die erzielten Gesamtscores als auch die Laufzeiten gegenübergestellt werden.

Über den hier gewählten Ansatz hinaus bietet die Methodik Potenzial für weiterführende Fragestellungen. So könnten zukünftig auch hybride Strategien betrachtet werden, bei denen mehrere Heuristiken kombiniert oder im Verlauf der Lösungssuche dynamisch gewechselt werden. Ebenfalls denkbar ist der Einsatz von Reinforcement-Learning-Verfahren, bei denen das System über wiederholte Rückmeldungen lernt, welche Heuristikabfolge bei bestimmten Graphstrukturen zu den besten Ergebnissen führt. Zunächst liegt der Fokus jedoch auf der systematischen Analyse der Leistungsfähigkeit adaptiver Metaheuristiken im TTDP.

Zum Abschluss der Einleitung folgt ein Überblick über den Aufbau dieser Arbeit. Kapitel 2 behandelt die theoretischen Grundlagen, insbesondere relevante Konzepte zu Heuristiken, Metaheuristiken und grundlegende Verfahren des maschinellen Lernens. In Kapitel 3 wird die Modellentwicklung beschrieben, wobei sowohl die Umsetzung der Heuristiken als auch die Gestaltung des Klassifikators im Fokus stehen. Kapitel 4 präsentiert die erzielten Ergebnisse, bevor diese in Kapitel 5 systematisch ausgewertet und verglichen werden. Kapitel 6 liefert schließlich einen Ausblick auf mögliche Weiterentwicklungen und zukünftige Forschungsansätze. Kapitel 7 schließt die Arbeit mit einem zusammenfassenden Fazit ab.

2. Theoretische Grundlagen

2.1. Ziel des Kapitels

Dieses Kapitel legt die theoretischen Grundlagen für diese Arbeit dar. Ziel ist es, die zentralen Begriffe, Notationen und konzeptionellen Zusammenhänge des Touristentour-Design-Problems (TTDP) klar zu definieren, sodass im weiteren Verlauf der Arbeit auf diesem gemeinsamen Verständnis aufgebaut werden kann.

Es werden die mathematischen Formulierungen des Problems, dessen Einordnung in verwandte kombinatorische Problemklassen sowie relevante Begriffe der Graphentheorie und eine konzeptionelle Übersicht über Heuristiken, Metaheuristiken und Algorithm-Selection-Ansätze präsentiert. Dabei liegt der Fokus auf Verständlichkeit der Theorie, wobei Formeln, Definitionen und begriffliche Abgrenzungen im Vordergrund stehen. Praktische Implementierungsdetails, konkrete Algorithmenimplementierungen, Datengenerierungen und die experimentelle Vorgehensweise werden im Kapitel *Modellentwicklung* behandelt.

2.2. Problemdefinition

Sei $G = (V, E)$ ein ungerichteter Graph mit Knotenmenge V und Kantenmenge $E = \{\{v_i, v_j\} \mid v_i, v_j \in V\}$. Jedem Knoten $v \in V$ ist ein nichtnegativer Score $s_v \in \mathbb{R}_{\geq 0}$ zugeordnet. Zu jeder Kante $\{u, v\} \in E$ gehört eine Distanz $d_{uv} \in \mathbb{R}_{\geq 0}$. Ferner sei $B \in \mathbb{R}_{>0}$ ein vorgegebenes Budget für die zulässige Gesamtdistanz.

Gesucht ist eine Sequenz von Knoten

$$\pi = (v_1, v_2, \ldots, v_k), \qquad k \in \mathbb{N},\ v_i \in V,$$

mit $v_1 = v_k$ und dem Kriterium

$$\{v_i, v_{i+1}\} \in E \quad \text{für alle } i = 1, \ldots, k-1,$$

damit alle Teilstrecken wohldefiniert sind. Mehrfachbesuche von Knoten sind zulässig, jedoch wird der Score eines Knotens höchstens einmal angerechnet. Sei

$$V(\pi) = \{\, v_i \mid 1 \leq i \leq k-1 \,\}$$

die Menge der mindestens einmal besuchten Knoten.

Die Gesamtkosten der Tour sind

$$C(\pi) = \sum_{i=1}^{k-1} d_{v_i v_{i+1}} \leq B, \tag{1}$$

und der gesammelte Score ist

$$S(\pi) = \sum_{v \in V(\pi)} s_v. \tag{2}$$

Formal ergibt sich somit das Optimierungsproblem

$$\max_{\pi} \; S(\pi) \quad \text{unter} \quad C(\pi) \leq B.$$

In dieser Arbeit wird das TTDP in einer vereinfachten Grundvariante betrachtet. Es wird angenommen, dass der zugrunde liegende Graph vollständig ist und symmetrische Distanzen erfüllt, so dass die Dreiecksungleichung gilt. Jeder Knoten ist mit einem deterministischen, zeitinvarianten Score versehen, der den Nutzen eines Besuchs widerspiegelt. Damit ergibt sich eine Instanzklasse, die wesentliche Eigenschaften realer Szenarien abbildet und zugleich eine mathematisch handhabbare Modellierung ermöglicht. Erweiterungen wie Zeitfenster, Kapazitätsrestriktionen oder stochastische Scores werden nicht berücksichtigt. Ebenso bleiben gerichtete und nicht-metrische Graphen außerhalb des Betrachtungsrahmens [11].

Die Entscheidungsvariante des TTDP, bei der zusätzlich ein Schwellwert S_0 vorgegeben ist und gefragt wird, ob es eine Tour π mit $C(\pi) \leq B$ und $S(\pi) \geq S_0$ gibt, ist in allgemeiner Form NP-vollständig (siehe 2.8). Daraus folgt unmittelbar, dass das zugehörige Optimierungsproblem NP-schwer ist. Exakte Verfahren sind daher für große Instanzen aufgrund des exponentiellen Wachstums der Lösungsräume praktisch nicht einsetzbar, was den Einsatz heuristischer und metaheuristischer Verfahren rechtfertigt.

2.3. Verwandte Probleme

Das TTDP steht in enger Beziehung zum *Orienteering Problem* (OP). Beim OP wird unter einer vorgegebenen Längen- beziehungsweise Budgetbegrenzung eine Teilmenge von Knoten ausgewählt und in eine Tour integriert, um die Summe der besuchten Gewinne zu maximieren. In der Literatur wird OP häufig als Oberbegriff für Routenprobleme mit Gewinnmaximierung für Budgetrestriktionen verwendet. Zahlreiche Varianten wie Team-Orienteering oder Versionen mit Zeitfenstern sind beschrieben und methodisch gut untersucht. Diese Problemfamilie ist der hier betrachteten TTDP-Variante konzeptionell am nächsten [26].

Vom klassischen *Traveling Salesman Problem* (TSP) unterscheidet sich TTDP grundlegend dadurch, dass nicht alle Knoten besucht werden müssen. Ein benachbartes Konzept ist das *Prize-Collecting TSP* (PCTSP), dass eine Rundreise mit Strafen für nicht besuchte Knoten modelliert und damit Kostenminimierung mit ‚Prämien' verknüpft, anstatt wie beim OP oder TTDP den Score unter einer Budgetrestriktion zu maximieren. PCTSP wird in der Literatur als Verallgemeinerung des TSP mit Prämien und Strafkosten gesehen und dient oft als Referenz für gewinnorientierte Tourenmodelle [9, 14].

Mehrere Touren führen von der Ein-Tour-Variante des OP oder TTDP zur Mehr-Tour-Variante, dem *Team Orienteering Problem* (TOP). Beim TOP werden mehrere Routen parallel geplant, jede Route unterliegt einer eigenen Längen- bzw. Budgetbeschränkung und jeder Knoten darf höchstens einmal insgesamt bedient werden. Das Ziel ist die Maximierung der Summe der einzelnen Gewinne. TOP ist damit die direkte Mehr-Routen-Verallgemeinerung des OP und gilt als kanonische Brücke zwischen gewinnmaximierenden Ein-Tour-Modellen und mehrtourigen Fahrzeugroutenproblemen [4, 26].

Zum klassischen *Vehicle Routing Problem* (VRP) ist TOP konzeptionell verwandt, unterscheidet sich jedoch in der Zielsetzung. Während VRP-Varianten typischerweise alle Kunden bedienen und primär Kosten minimieren, selektiert TOP nur eine vielversprechende Teilmenge von Knoten und maximiert die Summe der besuchten Gewinne unter Routenbudgets. In der Literatur wird TOP daher häufig als gewinnorientierter Verwandter innerhalb der VRP-Familie behandelt und bildet die Basis für Erweiterungen wie Zeitfenster (TOPTW) [4, 14, 24].

Als weitere nahe Verwandtschaft gilt das *Profitable Tour Problem* (PTP). Im Unterschied zum OP oder TTDP wird hier typischerweise nicht eine feste Budgetgrenze vorgegeben, sondern ein Nettonutzen optimiert, der Gewinne abzüglich Reisekosten umfasst. Damit verschiebt sich die Nebenbedingung in die Zielfunktion, die Auswahl- und Routenentscheidungen aber bleiben strukturell eng verwandt [6].

2.4. Relevante Konzepte der Graphentheorie

Im Folgenden werden die Begriffe der Graphentheorie eingeführt, die für die theoretische Einordnung des TTDP wesentlich sind. Die konkrete Operationalisierung als Merkmale mit Auswahl, Berechnung und empirischer Nutzung erfolgt später in der Arbeit.

Ausgangspunkt ist ein gewichteter, vollständiger und metrischer Graph $G = (V, E)$ mit Knotenmenge V und Kantenmenge E. Eine Tour ist eine endliche Knotenfolge $\pi = (v_1, \ldots, v_k)$, deren aufeinanderfolgende Knoten durch Kanten verbunden sind und deren Gesamtkosten die vorgegebene Budgetrestriktion erfüllen. Pfade und Zyklen sind die üblichen Spezialfälle. In der hier betrachteten Variante wird eine Rundreise vorausgesetzt, also $v_1 = v_k$ [7, 28].

Die Distanzen werden durch eine nichtnegative und symmetrische Funktion modelliert, die die Dreiecksungleichung erfüllt. Für Netze, die ursprünglich nicht vollständig sind, werden die paarweisen Distanzen als Längen kürzester Wege definiert. Bezeichnet $\mathrm{dist}_G(u, v)$ die Länge eines kürzesten Weges zwischen u und v, so entsteht durch Verwendung dieser Weglängen ein vollständiger, metrischer Distanzraum, der für die Formulierung des TTDP geeignet ist [20].

Für die strukturelle Charakterisierung einer Instanz sind Größe und Konnektivität zentral. Die Zahl der Knoten $|V|$ und der Kanten $|E|$ beeinflusst die kombinatorische Komplexität des Lösungsraums. Die ungerichtete Dichte

$$\delta(G) = \frac{2|E|}{|V|(|V| - 1)}$$

liefert ein normiertes Maß der Verbindungsintensität. In zusammenhängenden Graphen ist jedes Knotenpaar über einen Pfad erreichbar. Andernfalls beschränkt sich die Suche auf die betrachtete Zusammenhangskomponente [7, 28].

Neben globalen Größen sind charakteristische Längenskalen hilfreich. Die mittlere oder mediale Paardistanz $\bar{d}$ und die mittlere Distanz zum nächsten Nachbarn d_{nn} geben an, wie eng oder weit eine Instanz im Raum liegt. Daraus lässt sich eine dimensionslose Maßzahl für den Budgetfaktor ableiten,

$$\tau = \frac{B}{|V|\, d_{\mathrm{nn}}}.$$

Kleine Werte deuten darauf hin, dass das Budget überwiegend lokale Bewegungen zulässt. Große Werte erlauben weiträumige Touren. Solche Normierungen erleichtern Vergleiche zwischen Instanzen unterschiedlicher Größe und Skala [20].

Die lokale Struktur einer Instanz wird durch Clustering und Gemeinschaftsbildung beschrieben. Ein hoher lokaler Clusteringkoeffizient weist auf dichte Nachbarschaften hin und begünstigt Touren, die viele nahegelegene Knoten kombinieren. Schwaches Clustering erfordert häufiger größere Sprünge. Zentralitätsbegriffe liefern Hinweise auf Hubs und Engpässe. Dazu zählen Degree-Zentralität, Closeness-Zentralität, Betweenness-Zentralität und Eigenvektor-Zentralität. Eine heterogene Gradverteilung weist oft auf wenige stark vernetzte Knoten mit strategischer Bedeutung hin [20, 27].

Auch Eigenschaften der Kosten- und Score-Verteilung sind relevant. Eine hohe Varianz der Distanzen erschwert budgeteffiziente lokale Schritte. Eine stark konzentrierte Score-Verteilung begünstigt Strategien, die gezielt Regionen mit hohem Nutzen ansteuern. Die hier eingeführten Konzepte bilden die theoretische Sprache zur Beschreibung und Einordnung von TTDP-Instanzen. Auf ihre konkrete Nutzung als Merkmalskandidaten sowie auf die Auswahl spezifischer Kenngrößen wird in der Methodik zurückgekommen.

2.5. Heuristiken und Metaheuristiken

In dieser Arbeit werden vier Heuristiken eingesetzt, die unterschiedliche Suchprinzipien abdecken und gemeinsam eine Verbindung aus schneller Konstruktion, lokaler Verbesserung und kontrollierter Diversifikation bereitstellen. Alle Verfahren operieren auf zulässigen Touren, prüfen die Budgetbedingung in jedem Schritt und bewerten Kandidaten primär nach dem erzielten Score. Bei gleichem Score dient die Gesamtdistanz als Tie-Break.

Die *Greedy Heuristik* konstruiert eine Tour schrittweise. In jedem Schritt wird ein unbesuchter Knoten an der Position eingefügt, an der der Quotient aus hinzugefügten Score und zusätzlichen Kosten maximal ist. Die Einfügung erfolgt dort, wo die Mehrkosten minimal sind. Der Prozess endet, sobald keine weitere zulässige Einfügung möglich ist. Greedy liefert damit eine schnelle Referenzlösung und geeignete Starttouren für nachträgliche Verbesserungsverfahren [25].

Greedy Randomized Adaptive Search Procedure (GRASP) kombiniert eine zufallsbasierte Konstruktion mit einer anschließenden lokalen Suche. In der Konstruktionsphase ersetzt eine adaptive Kandidatenliste die reine Greedy Entscheidung, und aus dieser Liste wird zufällig gewählt, um die Lösung zu diversifizieren. Die anschließende lokale Suche verbessert die Tour bis zu einem stabilen Zustand. Mehrere unabhängige Starts erhöhen die Robustheit der Ergebnisse und führen typischerweise zu guten Lösungen bei moderatem Rechenaufwand [17].

Iterated Local Search (ILS) beginnt mit einer zulässigen Tour und führt eine lokale Suche aus, bis keine Verbesserung mehr möglich ist. Anschließend wird die Lösung gezielt gestört, etwa durch das Entfernen eines kurzen Segments mit anschließender erneuter Einfügung, und danach erneut lokal verbessert. Eine einfache Akzeptanzregel übernimmt verbesserte Lösungen. Auf diese Weise wechselt das Verfahren zwischen Stabilisierung in einem lokalen Optimum und dem Verlassen dieser Region. Kostenreduktionen schaffen dabei häufig Spielraum für weitere budgetkonforme Einfügungen [13].

Variable Neighborhood Search (VNS) arbeitet mit einer geordneten Familie von Nachbarschaften. Die Suche beginnt mit kleinen Veränderungen, schüttelt die aktuelle Tour auf und führt eine lokale Suche aus. Führt das zu einer Verbesserung, wird zur kleinsten Nachbarschaft zurückgekehrt. Bleibt die Verbesserung aus, wird zur nächstgrößeren Nachbarschaft gewechselt. Dadurch entsteht ein systematischer Wechsel zwischen Intensivierung in feinen Nachbarschaften und Diversifikation über größere Veränderungen [21].

Zusammen liefern die vier Verfahren eine schnelle Baseline, diversifizierte Konstruktionen, robuste Verbesserungen nach gezielten Störungen und eine strukturierte Erweiterung der Suche über mehrere Nachbarschaften hinweg. In der Kombination entsteht eine belastbare Grundlage, um das TTDP unter einer Budgetrestriktion mit deterministischen Scores effizient zu lösen.

2.6. Theorie der KI zur Algorithmenauswahl

Die Auswahl eines geeigneten Verfahrens für eine gegebene Instanz ist ein eigenes, gut untersuchtes Problem. Das klassische Modell von Rice beschreibt die Algorithmenauswahl als Abbildung von Instanzmerkmalen und Algorithmenräumen auf eine Leistungsgröße und zielt darauf, für jede Instanz den Algorithmus mit erwartet bester Leistung zu wählen. Üblicherweise wird dazu ein Lernmodell auf früheren Läufen trainiert, das die zu erwartende Güte der einzelnen Verfahren für neue Instanzen schätzt [22].

Für kombinatorische Such- und Optimierungsprobleme hat sich *per-instance* Auswahllogik etabliert. Aus einer Menge komplementärer Verfahren wird pro Instanz ein Kandidat bestimmt. Grundlage sind Instanzmerkmale, die die Struktur der Eingabe erfassen, sowie Leistungsdaten der Verfahren auf Trainingsinstanzen. Das Vorgehen lässt sich als Klassifikation, also die Vorhersage der Leistung aller Kandidaten und anschließende Wahl des Besten, formulieren. Eine aktuelle Übersicht für kombinatorische Probleme stellt die Vielfalt dieser Ansätze dar und zeigt den praktischen Nutzen, wenn Algorithmen unterschiedliche Stärken auf verschiedene Instanztypen haben [18].

Als methodischer Rahmen dient häufig ein *Algorithmusportfolio*. Ein Portfolio enthält mehrere, möglichst wenig zusammenhängende Verfahren. Ein Lernmodell entscheidet anhand von Merkmalen, welches Verfahren für die vorliegende Instanz gestartet wird. Prominent ist *SATzilla*, das Merkmalsräume über empirische Härtemodelle mit Vorhersagen verknüpft und zudem erste Lösungsschritte für sehr leichte Instanzen nutzt. Die resultierenden Systeme erreichen in Benchmarks Leistungen nahe einem Orakel, das rückblickend immer den jeweils besten Solver wählt [30].

Für die Ausgestaltung der Merkmalsseite ist wichtig, dass deren Berechnungskosten in der Gesamtrechnung berücksichtigt werden. In standardisierten Vergleichsumgebungen wie ASlib wird die Qualität von Auswahlsystemen deshalb gegen starke Baselines bewertet: den *Single Best Solver* (SBS) als bestes Einzelverfahren über alle Instanzen, den *Virtual Best Solver* (VBS) als Orakelobergrenze, sowie Kennzahlen wie mittlere Anzahl gelöster Instanzen und Fehlklassifikationsstrafe. Diese Benchmarks zeigen konsistent, dass lernbasierte Auswahl große Teile der Lücke zwischen SBS und VBS schließen kann, sofern geeignete Merkmale und Trainingsdaten vorliegen [2].

Ein zentraler Baustein sind empirische Leistungsmodelle, die Laufzeit, Lösungsqualität oder Erfolgswahrscheinlichkeit als Funktion von Instanzmerkmalen vorhersagen. Solche Modelle erlauben auch Kosten-Nutzen-Abwägungen zwischen Merkmalsberechnung, Vorlöseschritten und eigentlicher Lösungssuche. Verfahren wie Regressionswälder, Regularisierungsverfahren oder Gauß Prozesse sind in diesem Kontext verbreitet und können neben Instanzmerkmalen auch Algorithmusparameter einbeziehen [16].

Für das TTDP bedeutet es, dass die zuvor eingeführten graphbasierten Kenngrößen den Merkmalsraum bilden, etwa mit Größe und Dichte des Graphen, Budget, lokale Nachbarschaftsstruktur oder Streuung der Distanzen und Scores. Auf Basis solcher Merkmale kann ein lernendes Auswahlsystem zwischen Greedy, GRASP, ILS und VNS wählen und damit je Instanz die Suchdynamik aktivieren, die voraussichtlich den höchsten Score unter Einhaltung des Budgets liefert. Die Bewertung erfolgt gegen SBS und VBS sowie unter Einbezug der Merkmalskosten, sodass Auswahlentscheidungen nicht nur qualitativ plausibel, sondern auch rechnerisch wirtschaftlich sind [2, 18].

2.7. Bewertungs- und Vergleichsmetriken

Ziel ist ein klarer, vergleichbarer Maßstab für alle Verfahren. Primär wird der erreichte Score $S(\pi)$ einer zulässigen Tour π bewertet, wobei die Budgetbedingung $C(\pi) \leq B$ erfüllt sein muss. Bei gleichem Score wird die kürzere Tour bevorzugt, also die mit geringerer Gesamtdistanz $C(\pi)$.

Um Ergebnisse über verschiedene Instanzen vergleichbar zu machen, wird pro Instanz j eine relative Qualität verwendet. Sei

$$S_{\text{alg}}^{(j)} \quad \text{der von einem Verfahren auf Instanz } j \text{ erzielte Score.}$$

Als Referenz dient der pro Instanz beste beobachtete Wert

$$S_{\text{best}}^{(j)} = \max_{\text{über alle in dieser Arbeit betrachteten Verfahren}} S_{\text{alg}}^{(j)},$$

auch als *Virtual Best Solver (VBS)* bezeichnet. Die relative Qualität eines Verfahrens auf Instanz j ist dann

$$r^{(j)} = \frac{S_{\text{alg}}^{(j)}}{S_{\text{best}}^{(j)}} \in [0, 1],$$

und der *Abstand zum Bestwert* lautet

$$A^{(j)} = S_{\text{best}}^{(j)} - S_{\text{alg}}^{(j)}.$$

Für die Einordnung wird zusätzlich der *Single Best Solver (SBS)* als starke Baseline verwendet. Das ist das eine Verfahren, das im Mittel über alle Instanzen am besten abschneidet. Der VBS dient als obere Vergleichsgröße und zeigt, welches Qualitätsniveau bei perfekter Verfahrenwahl pro Instanz innerhalb des hier betrachteten Verfahrenspools erreichbar wäre [2, 18].

Zur relativen Einordnung werden die Within-δ-Raten aufgezeigt. Eine Instanz gilt als Within-δ, wenn

$$s_j(L) \geq (1 - \delta) \cdot S_{\text{best}}^{(j)}(L),$$

wobei L jeweils die Zeitlimits darstellt und $\delta \in \{0{,}1\,\%, 1\,\%, 5\,\%\}$ verwendet wird. Als komplementäre Verteilungsdarstellung werden Leistungsprofile auf Basis der Quotienten $q_j(L) = \frac{S_j(L)}{S^{(j)}_{\text{best}}(L)}$ genutzt. Die empirische Verteilungsfunktion der $q_j(L)$ beschreibt die Verteilung der VBS-Quoten.

Für Vergleiche mit dem SBS wird eine scorebasierte Dreiteilung auf Instanzebene verwendet. Es gilt *Win*, wenn $S_j(L)$ über dem SBS-Score liegt, *Tie*, wenn Gleichstand nach Score und Tie-Break vorliegt und *Loss* andernfalls. Die Nicht-Unterlegenheitsrate ist $p_{\geq \text{SBS}}(L) = \mathbb{P}[\text{Win} \vee \text{Tie}]$.

Die Klassifikationsgüte bezieht sich auf die Vorhersage der Heuristikklasse. Die *Accuracy* ist der Anteil korrekter Zuordnungen. Der *macro-F1* ist das arithmetische Mittel der klassenweisen F1-Werte und gewichtet alle Klassen gleich. *Confusion-Matrizen* stellen die gemeinsamen Häufigkeiten von wahrer und vorhergesagter Klasse dar und machen Verwechslungsmuster sichtbar [23].

Zur Erklärung von Merkmalsbeiträgen wird eine Permutationsauswertung genutzt. Für ein Merkmal x_k wird die Eingabe über Instanzen zufällig vertauscht, das Modell bleibt unverändert. Der Beitrag ergibt sich aus dem Güteabfall $\Delta M_k = M_{\text{original}} - M_{\text{permutiert}(x_k)}$ [3, 10]. Global wird die Accuracy betrachtet, klassenweise der F1 von der jeweiligen Klasse. Bei korrelierten Merkmalen können Beiträge auf austauschbare Signale verteilt sein.

Unsicherheit und Unterschiede werden nichtparametrisch beschrieben. Ein 95 %-Konfidenzintervall für einen Mittelwert wird per Bootstrap über Instanzen konstruiert und aus den Quantilen der Bootstrap-Verteilung des Schätzers gewonnen[8, 15]. Gepaarte Unterschiede in Abständen zum VBS werden mit dem Wilcoxon-Vorzeichenrangtest getestet. Die p-Werte werden über die fünf Zeitlimits schrittweise nach Holm korrigiert. Für binäre richtig-falsch-Vergleiche auf derselben Instanz, etwa gegenüber einer Referenzentscheidung, wird der McNemar-Test herangezogen [19, 29].

Die genaue Bildung der Referenzwerte $S^{(j)}_{\text{best}}$ sowie die Art der Aggregation über Instanzen werden im Kapitel zur *Modellentwicklung* und zum experimentellen Setup präzisiert.

2.8. Komplexität

Für die Bewertung der Lösbarkeit wird die Entscheidungsvariante des TTDP betrachtet. Gegeben sind ein vollständiger, metrischer Graph $G = (V, E)$ mit Distanzen d, nichtnegative Knoten-Scores s_v, ein Budget $B > 0$ und ein Zielwert S_0. Gesucht ist eine Rundtour π mit $v_1 = v_k$, deren Gesamtdistanz $C(\pi) \leq B$ bleibt und deren Score $S(\pi) \geq S_0$ erreicht. In Zeit $O(k)$ lassen sich $C(\pi)$ und $S(\pi)$ berechnen und mit B bzw. S_0 vergleichen, ebenso die Rundreise Bedingung $v_1 = v_k$ und die Einmalbesuchsbedingung prüfen ($k = |\pi|$). Damit liegt das Problem in NP.

Mit k wird die Zahl der unterschiedlichen besuchten Knoten der Tour bezeichnet. Für vollständige, metrische Graphen gilt stets $k \leq |V|$, da eine Tour nicht mehr verschiedene Knoten enthalten kann als vorhanden sind.

Die NP-Schwere folgt durch eine polynomielle Reduktion vom TSP-Entscheidungsproblem. Sei dazu eine TSP Instanz mit Schranke L gegeben: ‚Gibt es eine Hamiltonkreis mit Länge $\leq L$?'. Es wird eine TTDP-Instanz auf demselben Distanzraum konstruiert, wobei

$$s_v := 1 \; \forall v \in V, \qquad S_0 := |V|, \qquad B := L$$

gesetzt wird. Existiert im TSP eine Rundtour $\leq L$, so ist dieselbe Tour für das TTDP zulässig und erzielt wegen $s_v = 1$ den Score $S(\pi) = |V| \geq S_0$. Umgekehrt existiert im TTDP eine zulässige Rundtour mit $S(\pi) \geq |V|$, so wurde jeder Knoten mindestens einmal besucht. Liegen die Distanzen metrisch vor (z. B. euklidisch oder als Längen kürzester Wege), können etwaige Wiederholungen mittels Dreiecksungleichung ohne Mehrkosten abgekürzt werden, wodurch ein Hamiltonkreis mit Länge $\leq B = L$ entsteht. Damit ist die Entscheidungsvariante des TTDP NP-hart. Zusammen mit der Zugehörigkeit zu NP ist sie NP-vollständig [12].

2.9. Zusammenfassung

Dieses Kapitel hat die theoretischen Grundlagen für die weitere Arbeit gelegt. Zunächst wurde das Touristentour-Design-Problem (TTDP) als Optimierungsaufgabe auf einem gewichteten, vollständigen und metrischen Graphen formal definiert. Die betrachtete Variante verlangt eine Rundreise, verwendet deterministische, zeitinvariante Scores und unterliegt einer Budgetrestriktion für die Gesamtdistanz. Damit ist die Auswahl der zu besuchenden Knoten und ihre Einbettung in eine Tour zu bestimmen, sodass der Gesamtscore maximal wird. Die Entscheidungsvariante ist in allgemeiner Form NP-vollständig, woraus die NP-Schwere der Optimierung folgt und der Einsatz heuristischer Verfahren motiviert.

Für die Einordnung und spätere Merkmalsbildung wurden die relevanten graphentheoretischen Konzepte eingeführt. Dazu zählen Pfade und Zyklen, Distanzbegriffe über Längen kürzester Wege, Zusammenhang und Dichte sowie charakteristische Längenskalen wie mittlere Paardistanz und durchschnittliche Nachbarentfernung. Die daraus abgeleitete, dimensionslose Kenngröße der Budgetniveau erlaubt es, Instanzen unterschiedlicher Größe und Skala vergleichbar zu machen. Ergänzend wurden lokale Strukturmaße wie Clustering und gängige Zentralitäten genannt, die Hinweise auf dichte Regionen, Engpässe und strategisch bedeutsame Knoten liefern. Eigenschaften der Distanz- und Scoreverteilung wurden als weitere Einflussfaktoren für die Lösbarkeit und die Wirksamkeit von Suchoperationen herausgestellt.

Auf dieser Basis wurden die in der Arbeit verwendeten Heuristiken skizziert. Greedy liefert schnelle, budgetkonforme Starttouren. GRASP kombiniert eine randomisierte Konstruktion über Kandidatenlisten mit lokaler Suche und gewinnt dadurch robuste Lösungen über mehrere Starts. Iterated Local Search verbindet lokale Verbesserung mit gezielter Störung, um lokale Optima zu verlassen und erneute Verbesserungen zu ermöglichen. Variable Neighborhood Search organisiert die Suche über eine geordnete Familie von Nachbarschaften und wechselt systematisch zwischen kleinen und größeren Veränderungen. Alle Verfahren bewerten primär den erzielten Score und verwenden die Distanz als Sekundärkriterium.

Schließlich wurde die Theorie der KI-gestützten Algorithmenauswahl umrissen. Per-instance Auswahlmodelle verknüpfen Instanzmerkmale mit einem Portfolio von zueinander verschiedenen Verfahren, um pro Instanz die aussichtsreichste Heuristik zu wählen. Für die Bewertung

dienen Bezugsgrößen wie der Single Best Solver und der Virtual Best Solver sowie standardisierte Metriken, wobei die Kosten der Merkmalsberechnung zu berücksichtigen sind.

Die genannten Definitionen, Konzepte und Verfahren bilden den theoretischen Rahmen für die nachfolgenden Kapitel. Dort werden die Heuristiken präzisiert und parametrisiert, die Merkmale angewandt, das Auswahlsystem konstruiert und die empirische Evaluierung auf geeigneten Instanzen durchgeführt.

3. Modellentwicklung

3.1. Zielsetzung und Annahmen

Ziel dieses Kapitels ist es, ein lauffähiges Modell für das hier betrachtete TTDP aufzubauen und die ausgewählten Verfahren unter gleichen Bedingungen zu untersuchen. Im Mittelpunkt stehen vier Heuristiken: Greedy, GRASP, ILS und VNS. Ziel ist es herausfinden, welche dieser Verfahren für die gewählten Instanzen den höchsten Score erreicht, wie stabil die Ergebnisse über mehrere Läufe sind und welche Bausteine der lokalen Suche besonders viel bewirken. Zusätzlich soll geprüft werden, ob sich durch eine einfache, merkmalbasierte Auswahl je Instanz ein Vorteil gegenüber einem festen Einzelverfahren erzielen lässt. Die Bewertung erfolgt später mit einem einheitlichen Zeitlimit und den im Theorieteil definierten Kennzahlen.

Das Modell folgt der in den Grundlagen beschriebenen Problemfassung. Es liegt ein ungerichteter Graph mit nichtnegativen Kantenlängen vor. Gesucht ist eine Rundtour, die beim Startknoten endet. Aufeinanderfolgende Knoten müssen über eine Kante verbunden sein, damit alle Teilstrecken wohldefiniert sind. Die Tour darf das vorgegebene Budget für die Gesamtdistanz nicht überschreiten. Jeder Knoten besitzt einen festen Score. Mehrfachbesuche sind erlaubt, der Score eines Knotens wird aber höchstens einmal angerechnet. Als Zielfunktion dient der Gesamt erzielte Score. Bei gleichem Score wird die kürzere Tour bevorzugt. Damit ist eindeutig festgelegt, wie Lösungen konstruiert, geprüft und verglichen werden.

Für die Distanzen wird ein metrischer Distanzraum verwendet. Ist der Graph nicht vollständig, wird die Distanz zwischen zwei Knoten als Länge eines kürzesten Weges im Graphen definiert. Dadurch entsteht eine vollständige und symmetrische Distanzfunktion. Diese Annahme erleichtert sowohl die korrekte Budgetprüfung als auch die Verbesserungen in der lokalen Suche. Erweiterungen wie Zeitfenster, Kapazitäten, gerichtete Kanten oder stochastische Scores werden in diesem Kapitel nicht berücksichtigt, da der Fokus auf einer klaren Grundvariante liegt, die sich gut auswerten lässt und als Basis für die Verfahren dient.

Operativ wird eine einfache Tourrepräsentation als Knotenfolge verwendet. Die Budgetprüfung stellt sicher, dass nach jedem Schritt der Rückweg zum Start eingeschlossen ist. Für alle Verfahren gelten dieselben Datenstrukturen. Die genaue Ausgestaltung der Konstruktionsschritte, der Nachbarschaften in der lokalen Suche sowie der Parameter wird in den folgenden Abschnitten beschrieben. Die Regeln für das Evaluationsprotokoll mit Zeitlimit, Seeds und Referenzwerten folgen gesammelt in einem eigenen Abschnitt.

3.2. Instanzbasis und Datenaufbereitung

3.2.1. Quellen der Instanzen

Die Experimente basieren auf 3 500 synthetischen Instanzen. Die Generierung orientiert sich an den Chao Instanzklassen[1], wurde jedoch auf die hier verwendete TTDP-Form angepasst und um Strukturmuster wie Score-Hotspots erweitert. Die Chao Klassen liegen in einer Einzel- und in einer Team-Variante vor, welche sich beide auf unserer Rundtour-Fassung abbilden lassen, in

[1] `https://www.mech.kuleuven.be/en/cib/op`

der das Depot als Start und Ziel gesetzt wird. Auf die originalen Chao Instanzen wird bewusst verzichtet, da ihre Knotenzahl gering ist und damit wenig Varianz bietet. Die synthetischen Instanzen decken unterschiedliche Strukturprofile ab und bilden so eine repräsentative Basis für die Auswertung.

3.2.2. Knotenlage und Distanzen

Die synthetischen Instanzen bestehen aus Koordinaten für Depot und Points of Interest, die entweder gleichmäßig verteilt oder um wenige Zentren geclustert sind. Die Distanzen werden beim Einlesen aus den Koordinaten als euklidischer Abstand auf dem vollständigen Graphen berechnet. Dadurch sind die Kosten symmetrisch.

3.2.3. Instanzgrößen und Budgetniveau

Die Auswertung umfasst mehrere Größenordnungen von $|V|$, damit kleine und mittlere Fälle abgedeckt sind. Das Budget wird entweder vorher festgelegt oder über eine einfache Kenngröße kalibriert. Dazu wird die durchschnittliche Distanz zum nächsten Nachbarn d_{nn} bestimmt und

$$B \;=\; \tau \cdot |V| \cdot d_{\mathrm{nn}}$$

gesetzt. Der Parameter τ steuert den Budgetfaktor. Kleine Werte begünstigen lokale Touren. Größere Werte erlauben weiträumige Bewegungen.

3.2.4. Scores

Für synthetische Instanzen stammen die Scores aus einem kleinen ganzzahligem Intervall. In einzelnen Instanzen werden Hotspots modelliert, in denen Knoten in dichten Regionen im Mittel höhere Werte besitzen. Der Score eines Knotens wird in jeder Tour höchstens einmal angerechnet.

3.2.5. Datenformat und Einlesen

Die Instanzen liegen in einer kompakten Textdatei vor. Die Datei beginnt mit der Zeile `n <anzahl der knoten>`. In der zweiten Zeile steht `m 1`. Direkt darunter folgt die Budgetzeile `tmax <wert>`. Anschließend steht pro Knoten eine Zeile mit `x y score`. Die erste Knotenzeile bildet den Startpunkt/Endpunkt und erhält in der Regel Score 0. Beim Einlesen werden die Distanzen aus den Koordinaten als euklidische Längen auf dem vollständigen Graphen berechnet.

3.2.6. Reproduzierbarkeit

Alle Instanzen besitzen eindeutige IDs und dokumentierte Zufallsseeds. Für die späteren Auswahlexperimente wird der Instanzsatz in feste Teile separiert, damit Training und Test getrennt bleiben. Die genutzten Zeitlimits und die Anzahl der Läufe pro Instanz sind im Evaluationsprotokoll festgelegt.

3.3. Konstruktionsphase

3.3.1. Ausgangspunkt und Notation

Die Konstruktion arbeitet auf einer Rundtour, die am selben Knoten startet und dort endet. Zu Beginn besteht die Tour nur aus dem Startknoten, also (v_0, v_0). Bei jeder Einfügung prüfe ich die Budgetbedingung mit Rückweg zum Startknoten. Der Score eines Knotens wird nur einmal gezählt. Mehrfachbesuche sind zwar erlaubt, bringen aber keinen zusätzlichen Score. Deshalb versucht die Konstruktion, vor allem neue Knoten aufzunehmen.

3.3.2. Einfügekosten und Score-Zuwachs

Für einen Kandidatknoten w wird zunächst die beste Einfügeposition in der aktuellen Tour ermittelt. Wird w zwischen zwei aufeinanderfolgende Knoten v_i und v_{i+1} eingefügt, entstehen zusätzliche Kosten

$$\Delta C(v_i, w, v_{i+1}) = d(v_i, w) + d(w, v_{i+1}) - d(v_i, v_{i+1}).$$

Der Score Zuwachs wird auf s_w gesetzt, falls w bisher nicht in der Tour enthalten ist. Andernfalls ist der Zuwachs 0. Für jeden Kandidaten wird die Position mit minimalen ΔC genutzt, die gleichzeitig die Budgetgrenze $t_{\max}$ nicht verletzt. Falls es keine zulässige Position gibt, scheidet der Kandidat in diesem Schritt aus.

3.3.3. Greedy Konstruktion

Die Greedy Variante wählt in jedem Schritt den Kandidaten mit dem besten Verhältnis aus Nutzen und Aufwand. Konkret werden zulässige Einfügungen über den Quotienten

$$q(w) = \frac{s_w}{\Delta C^*(w)}$$

bewertet, wobei $\Delta C^*(w)$ die minimalen Mehrkosten über alle Einfügepositionen für w sind. Bei Gleichstand werden zuerst die geringeren Mehrkosten bevorzugt und danach der höhere Score. Der gewählte Knoten wird an der zugehörigen Bestposition eingefügt. Der Prozess endet, wenn kein Kandidat mehr zulässig ist oder alle verbleibenden Kandidaten keinen positiven Beitrag liefern. Das Ergebnis ist eine schnelle, nachvollziehbare Tour.

3.3.4. GRASP Konstruktion

GRASP baut auf der Greedy Idee auf, führt aber Zufall ein und bewertet nicht nur den einzelnen Knoten, sondern den Nettoeffekt der gesamten Aktion. Für jeden noch unbesuchten Knoten w wird zunächst die Einfügeposition mit minimalen Mehrkosten ΔC bestimmt. Aus der neuen Tourlänge T und der aktuellen Länge L ergibt sich der effektive Aufwand $T - L$. Der zugehörige Gewinn ist die Score-Differenz $\Delta S = S_{\text{neu}} - S_{\text{alt}}$. Die Bewertung erfolgt über den Quotienten

$$q(w) \;=\; \frac{\Delta S}{\max(T - L, \varepsilon)},$$

wobei ε eine kleine Toleranz gegen Rundungsfehler ist. Aus allen zulässigen Kandidaten wird eine *Restricted Candidate List* (RCL) gebildet, die alle Optionen mit $q(w) \geq \alpha \cdot q_{\max}$ enthält. Der nächste Knoten wird zufällig aus dieser RCL gewählt und an der zugehörigen Bestposition eingefügt. Führt die direkte Einfügung zum Überschreiten des Budgets, wird eine kleine Reparatur versucht, bei der ein kurzes Segment entfernt wird, solange dadurch eine zulässige und gegenüber der aktuellen Tour vorteilhafte Lösung entsteht. Nach jeder Einfügung erfolgt eine kurze 2-opt (vgl. 3.4.1.1) Ausführung. Eine neue Tour wird nur akzeptiert, wenn der Score steigt oder bei gleichem Score die Gesamtlänge sinkt. Der Ablauf wiederholt sich, bis keine zulässige Einfügung mehr möglich ist oder ein Zeitlimit erreicht ist. Mehrere unabhängige Starts mit unterschiedlichen Zufallszahlen liefern robuste Touren.

3.3.5. Kriterien von Greedy und GRASP

Die Greedy Konstruktion in dieser Arbeit wählt in jedem Schritt die *Best Insertion* mit maximalem Verhältnis $\frac{s_w}{\Delta C(w)}$. GRASP bewertet dagegen den Nettoeffekt der gesamten Aktion $\frac{\Delta S}{T - L}$ nach Best Insertion und kleiner Reparatur. Das macht die Auswahl besonders bei knappen Budgets stabiler, weil teure Rückwege und Verdrängungseffekte direkt in die Bewertung eingehen.

3.3.6. Abbruch und Gültigkeit

Nach jeder Einfügung wird die Gesamtdistanz gegen $t_{\max}$ geprüft. Vergleiche erfolgen mit einer kleinen Toleranz, damit Rundungsfehler keine Rolle spielen. Eine Konstruktion ist abgeschlossen, wenn alle verbleibenden Einfügungen die Budgetgrenze überschreiten oder keinen Score mehr bringen. Die so erzeugten Touren sind zulässig und dienen als Eingabe für die lokale Suche und den Metaheuristiken *Iterated Local Search* und *Variable Neighborhood Search* in den folgenden Abschnitten (siehe 3.4, 3.5).

3.4. Lokale Suche und Nachbarschaften

Die in der Konstruktionsphase erzeugten Touren sind gültig, aber in vielen Fällen noch nicht die beste erreichbare Lösung. Die lokale Suche dient dazu, solche Touren systematisch zu verbessern. Grundlage ist weiterhin dieselbe Bewertung: Zunächst zählt der erreichte Score,

bei gleichem Score wird die Tour mit der geringeren Gesamtdistanz bevorzugt. Jede Änderung muss die Budgetgrenze $t_{\max}$ einhalten. Vergleiche gegen das Budget erfolgen mit einer kleinen Toleranz ε, damit Rundungsfehler nicht zu ungültigen Touren führen.

3.4.1. Reihenfolgeändernde Nachbarschaften

In einem ersten Schritt werden Bewegungen eingesetzt, die nur die Reihenfolge der bereits besuchten Knoten verändern. Die Menge der Knoten bleibt gleich. Der Score ändert sich daher nicht. Ziel dieser Bewegungen ist es, eine vorhandene Tour so umzuordnen, dass dieselben Knoten mit weniger Weglänge besucht werden. Diese Art von Nachbarschaft ist vergleichsweise günstig zu prüfen und liefert oft schon eine deutliche Verbesserung nach der Konstruktion.

3.4.1.1. 2-opt 2-opt ist ein sehr häufig verwendeter Operator für Tourenprobleme. Die Idee ist einfach. Wenn sich in einer Tour zwei Kanten kreuzen oder ein Abschnitt einen deutlichen Umweg macht, lässt sich die Tour oft verkürzen, indem man einen Teil der Tour umkehrt. Formal werden alle Indexpaare (i, j) mit $1 \leq i < j \leq m-2$ betrachtet. Zu diesem Paar gehören die beiden Kanten (v_{i-1}, v_i) und (v_j, v_{j+1}). Diese beiden Kanten werden entfernt und durch die Kanten (v_{i-1}, v_j) und (v_i, v_{j+1}) ersetzt. Damit ist das Segment $(v_i, v_{i+1}, \ldots, v_j)$ umgedreht. Die Änderung der Tourlänge lautet

$$\Delta = d(v_{i-1}, v_j) + d(v_i, v_{j+1}) - d(v_{i-1}, v_i) - d(v_j, v_{j+1}).$$

Ist $\Delta < 0$, wird die Umkehrung übernommen. Anschließend wird mit der neuen Tour weitergesucht. Dieser Vorgang wird wiederholt, bis kein Paar mehr gefunden wird, das eine echte Verkürzung bringt [5].

3.4.1.2. Relocate Bei der Relocate-Bewegung wird genau ein innerer Knoten an eine andere Stelle der Tour verschoben. Dazu wird der Knoten zunächst entfernt. Die entstehende Lücke wird durch direkte Verbindung der beiden umliegenden Knoten geschlossen. Anschließend wird der entfernte Knoten an jeder möglichen Position wieder eingefügt. Für jede Einfügeposition wird die entstehende Mehrlänge berechnet. Die Position mit den geringsten Mehrkosten wird ausgewählt. Diese Variante wird übernommen, wenn die daraus entstehende Tour noch innerhalb von $t_{\max}$ liegt und nicht länger ist als die Tour vor der Verschiebung. Relocate ist besonders dann wirksam, wenn die Konstruktion einen Knoten zu früh eingefügt hat und er deshalb in einer zu teuren Nachbarschaft steht.

3.4.1.3. Swap Swap vertauscht die Position zweier innerer Knoten. Auch hier werden alle möglichen Paare von Positionen innerhalb der Tour betrachtet, die nicht der Startknoten sind. Für jedes Paar wird die Tourlänge neu berechnet. Liefert der Tausch eine kürzere Tour, wird er übernommen. Swap ist sehr billig zu prüfen und beseitigt kleine lokale Fehlanordnungen, bei denen zwei Knoten in der Tour zwar korrekt enthalten sind, aber in umgekehrter Reihenfolge effizienter besucht werden können.

3.4.1.4. Blockverschiebung In geclusterten Instanzen treten Knoten oft in kleinen räumlich nahen Gruppen auf. Werden diese Knoten in der Konstruktion nacheinander eingefügt, ist das günstig. Durch spätere Verbesserungen oder Störungen können diese Gruppen jedoch auseinandergerissen werden. Um das rückgängig zu machen, wird eine Blockverschiebung eingesetzt. Dabei werden zwei oder drei aufeinander folgende Knoten als zusammenhängender Block aus der Tour entfernt und an einer anderen Stelle wieder eingefügt. Dadurch bleiben räumlich zusammengehörige Knoten beieinander. Auch hier wird die neue Tour nur übernommen, wenn sie zulässig ist und die Distanz nicht verschlechtert.

3.4.2. Mengenändernde Nachbarschaften

Reihenfolgeänderungen reichen nicht immer aus. Häufig ist es so, dass nach einer Distanzverkürzung wieder Budget frei wird. Dieses Budget kann genutzt werden, um weitere, bisher unbesuchte Knoten aufzunehmen. Deshalb werden zusätzlich Nachbarschaften verwendet, die die Menge der besuchten Knoten verändern. Die Kosten werden dabei genau so berechnet wie in der Konstruktion.

3.4.2.1. Add Mit Add wird ein bisher unbesuchter Knoten in die Tour eingefügt. Dazu wird für jede mögliche Einfügeposition zwischen zwei aufeinander folgenden Knoten die entstehende Mehrlänge berechnet. Die kleinste dieser Mehrlängen bestimmt die beste Einfügeposition. Wird der neue Knoten dort eingefügt, entsteht eine neue Tour. Diese Tour wird nur übernommen, wenn sie das Budget einhält und der neue Knoten einen positiven Score besitzt, der bisher noch nicht in der Tour enthalten war.

3.4.2.2. Drop Mit Drop wird ein Knoten aus der Tour entfernt. Das lohnt sich vor allem dann, wenn ein Knoten mehrfach besucht wird, der Score aber nur einmal gezählt wird. In diesem Fall belegt der zweite Besuch nur Budget und trägt nichts zum Ergebnis bei. Durch das Entfernen wird Budget frei, das anschließend für Add oder für komplexere Verbesserungen genutzt werden kann. Drop wird ebenfalls nur übernommen, wenn die resultierende Tour gültig ist.

3.4.2.3. Replace Replace kombiniert Add und Drop. Zunächst wird ein bereits enthaltener, wenig lohnender Knoten entfernt. Anschließend wird ein besser bewerteter, bisher unbesuchter Knoten an seiner besten Position eingefügt. Die Bewegung wird nur akzeptiert, wenn der Score dadurch steigt oder gleich bleibt und die Budgetbedingung erfüllt bleibt. Replace ist sinnvoll, wenn die Tour bereits viele Knoten enthält, aber einige davon nur sehr kleine Scores besitzen.

3.4.3. Suchstrategie und Abbruch

Die lokale Suche in dieser Arbeit basiert auf dem Prinzip der Erstverbesserung. Die Nachbarschaften werden in einer festen Reihenfolge ausprobiert. Typischerweise beginnt die Suche mit 2-opt, weil sich damit sehr schnell unnötige Umwege entfernen lassen. Danach folgen Relocate

und Swap, um die Reihenfolge zu verfeinern. Erst danach werden mengenändernde Bewegungen wie Add oder Replace betrachtet, weil sie auf einer bereits bereinigten Tour meist bessere Ergebnisse liefern. Sobald eine Bewegung eine Verbesserung ergibt, wird die neue Tour sofort übernommen und die Suche beginnt mit dieser Tour erneut. Auf diese Weise wird vermieden, dass zu viele Varianten einer Tour parallel gehalten werden müssen. Die Suche wird beendet, wenn in einem vollständigen Durchlauf über alle gewählten Nachbarschaften keine Verbesserung mehr gefunden wird oder wenn das für die Heuristik gesetzte Zeitlimit erreicht ist.

3.4.4. Einordnung in die Metaheuristiken

Die hier beschriebene lokale Suche bildet die gemeinsame Grundlage für die später eingesetzten Metaheuristiken. In Iterated Local Search wird nach jeder Störung der Tour genau diese lokale Suche aufgerufen, um aus der gestörten Lösung wieder eine stabile Lösung zu machen. In Variable Neighborhood Search wird nach einem Shake ebenfalls lokal verbessert. Dort werden die oben genannten Nachbarschaften jedoch in mehreren Stufen durchlaufen. GRASP nutzt dieselben Operatoren nach der zufallsbasierten Konstruktion, um eine gefundene Tour auf ein lokales Optimum zu bringen. Auch wenn die konkrete Reihenfolge und die Auswahl der Operatoren je Verfahren leicht variiert, bleibt die Grundidee identisch. Zuerst wird eine zulässige Tour erzeugt, dann werden systematisch kleine Veränderungen ausprobiert, und es wird nur das übernommen, was entweder mehr Score liefert oder denselben Score zu geringeren Kosten erreicht.

3.5. Metaheuristiken

Die im theoretischen Teil beschriebenen Verfahren werden in diesem Abschnitt in einer einheitlichen, auf das TTDP zugeschnittenen Form eingesetzt. Ziel ist es, auf allen Instanzen vergleichbare Läufe zu erhalten, sodass die späteren Auswertungen und die Algorithmenauswahl auf derselben Datenbasis aufbauen können. Alle Verfahren arbeiten auf zulässigen Rundtouren, prüfen jede Veränderung gegen das Budget $t_{\max}$ und verwenden dieselbe Bewertungsregel.

Für GRASP wird eine konstruktive Phase mit beschränkter Kandidatenliste genutzt. Die Kandidaten werden nach dem Verhältnis aus zusätzlichem Score und effektiven Einfügekosten sortiert. Die Auswahl aus den besten Kandidaten erfolgt zufällig. Jede so konstruierte Tour wird anschließend mit 2-opt und einfachen Einfüge-/Entfernungsoperationen lokal verbessert. Die Zahl der Konstruktionsschritte und die Zahl der Lokalsuchrunden sind begrenzt, außerdem wird in jeder Iteration ein Zeitlimit geprüft.

Die ILS-Variante beginnt mit einer konstruktiv erzeugten Tour und führt darauf die lokale Suche aus, bis kein Verbesserungsschritt mehr gefunden wird. Danach wird die Tour leicht gestört, indem ein Teilsegment entfernt oder versetzt wird. Auf die gestörte Tour wird erneut die lokale Suche angewendet. Verbessert sich der Score, wird die neue Tour übernommen. Andernfalls bleibt die bisher beste Tour bestehen. Durch das wiederholte Wechselspiel aus Störung und Verdichtung können mehrere Bereiche der Instanz getestet werden.

Das VNS-Verfahren nutzt dieselben Grundbewegungen wie die lokale Suche, ordnet sie aber in mehreren Nachbarschaften unterschiedlicher Größe an. Nach jeder Verbesserung wird wieder in der kleinsten Nachbarschaft weitergesucht. Führt eine Nachbarschaft zu keiner Verbesserung,

wird die nächstgrößere ausprobiert. Die Shake-Operationen sind so gewählt, dass die Rundtour erhalten bleibt und das Budget nicht dauerhaft verletzt wird. Auch hier begrenzt ein Zeitlimit die Gesamtlaufzeit.

3.6. Parameterauswahl

Für die empirische Auswertung und für die spätere Algorithmenauswahl müssen alle betrachteten Heuristiken unter vergleichbaren Bedingungen laufen. Dazu werden die Parameter der Verfahren einmal festgelegt und anschließend für alle Instanzen beibehalten. Auf diese Weise lassen sich Unterschiede in den Ergebnissen auf die Instanzstruktur und auf das Verfahren selbst zurückführen und nicht auf zufällig gewählte Einstellungen.

3.6.1. Zeitbudgets

Alle Verfahren werden mit denselben fünf Laufzeitgrenzen ausgeführt,

$$t \in \{1\text{s}, 5\text{s}, 10\text{s}, 20\text{s}, 40\text{s}\}.$$

Die kurzen Laufzeiten zeigen das Verhalten der reinen Konstruktion, die längeren Laufzeiten erlauben auch mehrere Runden lokaler Suche bzw. mehrere Störschritte. In allen Implementierungen wird das Zeitlimit in den äußeren Schleifen geprüft, sodass die reale Laufzeit nah am vorgegebenen Wert bleibt. Zu jedem Lauf werden Algorithmus, Instanz, Zeitlimit, erzielter Score, Distanz bzw. verbleibendes Budget und tatsächliche Laufzeit gespeichert.

3.6.2. Gemeinsame Bewertung

Unabhängig vom Verfahren wird jede Tour nach derselben Regel bewertet. Zuerst wird der erreichte Gesamtscore betrachtet. Erzielen mehrere Touren denselben Score, wird die Tour mit der geringeren Gesamtdistanz bevorzugt. Die Budgetbedingung wird mit einer kleinen Toleranz ε geprüft, um Rundungsfehler bei euklidischen Distanzen abzufangen. Da alle Heuristiken diese Regel verwenden, können ihre Ergebnisse direkt miteinander verglichen werden.

3.6.3. GRASP

Für GRASP wird eine konstruktive Phase mit beschränkter Kandidatenliste eingesetzt. Die Kandidaten werden nach dem Verhältnis aus zusätzlichem Score und effektiven Einfügekosten sortiert. Aus den besten Kandidaten wird zufällig ausgewählt. Der Parameter α wird auf $0,2$ gesetzt, sodass die Konstruktion überwiegend zielgerichtet bleibt, aber noch genug Diversität erzeugt. Die Zahl der Konstruktionsschritte wird auf ein Vielfaches der Knotenzahl begrenzt, ebenso die Zahl der Lokalsuchrunden. Dadurch wächst die Rechenzeit nicht unkontrolliert mit der Instanzgröße.

3.6.4. Iterated Local Search

Die ILS-Variante nutzt die in Abschnitt zur lokalen Suche beschriebenen Operatoren. Nach jedem lokalen Optimum wird die Tour leicht gestört, durch ein Entfernen oder Versetzen eines kurzen Segments und anschließend erneut verbessert. Die maximale Zahl an aufeinander folgenden Iterationen ohne Verbesserung wird begrenzt. Wird diese Grenze erreicht oder läuft das Zeitbudget ab, wird der aktuelle Lauf beendet. Damit bleibt ILS auch bei großen Instanzen innerhalb der vorgegebenen Zeit.

3.6.5. Variable Neighborhood Search

Für VNS werden zwei Nachbarschaftsebenen verwendet. Die erste Ebene enthält einfache Bewegungen wie Verschieben und Vertauschen einzelner Knoten. Die zweite Ebene enthält gröbere Änderungen durch Segmentverschiebungen. Nach jedem Shake wird die lokale Suche aufgerufen. Führt eine Ebene zu einer Verbesserung, wird zur kleinsten Ebene zurückgesetzt. Führt sie zu keiner Verbesserung, wird die nächste Ebene ausprobiert. Auch hier sorgen Zeitprüfungen in der Hauptschleife dafür, dass das Verfahren die vorgegebene Laufzeit nicht deutlich überschreitet.

3.6.6. Zufall und Seeds

GRASP, ILS und VNS enthalten zufällige Entscheidungen. Um diese Variabilität sichtbar zu machen, werden pro Instanz und Zeitbudget mehrere Läufe mit denselben wenigen Seeds (z.B. 0, 1, 2) durchgeführt. So lassen sich Unterschiede durch Zufall vom eigentlichen Verfahren trennen, ohne die Parameter zwischen den Instanzen ändern zu müssen.

3.7. Algorithmenauswahl

Die bisherigen Abschnitte beschreiben die vier verwendeten Heuristiken, ihre Parameter sowie die Instanzmerkmale. Darauf aufbauend wird nun ein Auswahlsystem beschrieben, das für eine neue TTDP-Instanz automatisch eines der Verfahren startet. Grundlage ist die *per-instance*-Algorithmenauswahl. Für jede Instanz soll das Verfahren gewählt werden, das unter den festgelegten Laufzeiten voraussichtlich den höchsten Score liefert.

3.7.1. Zielsetzung

Ziel der Algorithmenauswahl ist eine Abbildung

$$\phi(x) \to a,$$

wobei x eine Instanz beschreibt, $\phi(x)$ der daraus berechnete Merkmalsvektor ist und $a \in \{\text{greedy}, \text{grasp}, \text{ils}, \text{vns}\}$ das ausgewählte Verfahren. Als obere Schranke dient ein virtueller Läufer, der für jede Instanz rückblickend einfach den besten der tatsächlich beobachteten Läufe nimmt.

3.7.2. Merkmalsraum

Als Eingabe werden die pro Instanz berechneten Merkmale verwendet. Dazu gehören u.a. eine Instanz-ID, Erzeugungseinstellungen, die Zahl der Knoten, Gradstatistiken (Minimum, Maximum, Mittelwert), Angaben zu Zusammenhangskomponenten, der mittlere Clustering-Wert, geometrische Kenngrößen der konvexen Hülle sowie dichte- und MST-basierte Größen. Besonders wichtig ist eine normierte Budgetgröße pro Knoten. Diese Merkmale liegen für alle Instanzen in gleicher Form vor und können ohne weitere Schritte in das Modell übernommen werden.

3.7.3. Zielvariablen

Für jede Instanz und jedes Zeitbudget liegen mehrere Läufe vor, jeweils für alle vier Verfahren und mehrere Seeds. Aus diesen Läufen wird genau ein Label abgeleitet. Zuerst wird der höchste beobachtete Score bestimmt. Alle Läufe, die diesen Score erreichen, kommen in die engere Wahl. Unter diesen wird die Tour mit der geringeren Distanz ausgewählt. Das zugehörige Verfahren bildet das Ziel für die Klassifikation.

3.7.4. Lernverfahren

Für die Auswahl wird ein Random-Forest-Klassifikator eingesetzt. Das Modell kann mit den gemischten Merkmalen umgehen, benötigt wenig Vorverarbeitung und liefert zusätzlich Hinweise auf wichtige Merkmale. Trainiert wird auf Instanzebene. Jede Instanz erscheint genau einmal mit ihrem Merkmalsvektor und dem abgeleiteten besten Verfahren. So wird verhindert, dass zufällige Unterschiede einzelner Läufe das Training verzerren.

3.7.5. Auswertung

Die Auswahl wird gegen zwei Baselines bewertet. Die erste ist der SBS, also dasjenige der vier Verfahren, das im Mittel über alle Instanzen am besten abschneidet. Die zweite ist der VBS, der für jede Instanz den tatsächlich besten beobachteten Lauf nimmt. Liegt die Auswahl klar über dem SBS, nutzt sie die Instanzmerkmale sinnvoll. Bleibt jedoch ein größerer Abstand zum VBS, können weitere Merkmale oder feinere Verfahrensvarianten ergänzt werden.

3.8. Modellvarianten

Es werden zwei Modellvarianten der Algorithmenauswahl verwendet. In der *per-Limit* Variante wird für jedes Zeitlimit ein eigenes Random-Forest-Modell trainiert, das ausschließlich Instanzen dieses Limits sieht. Im *globalen* Setting lernt ein einzelner Random Forest über alle Limits, wobei das Zeitlimit als Merkmal `limit_s` einfließt. Zielvariable und Featurebasis sind in beiden Varianten identisch und folgen der Bestimmung der besten Heuristik pro Instanz und Limit nach Score mit definiertem Tie-Break über Distanz und Zeit.

3.9. Verwendete Bibliotheken und Tools

Für die Modellentwicklung und die Auswertung werden bewährte Python-Bibliotheken genutzt. Die Klassifikation erfolgt mit dem Random Forest aus `scikit-learn`. Die Signifikanztests basieren auf `SciPy` und `statsmodels`. Tabellen- und Vektoroperationen laufen über `pandas` und `NumPy`. Die Abbildungen sind mit `matplotlib` erstellt. Die graphbasierten Merkmalsberechnungen greifen auf eigene Hilfsfunktionen auf Basis von `NumPy` und `SciPy` zurück.

Tabelle 1: Eingesetzte Bibliotheken und zentrale Aufrufe

Bibliothek	Einsatz im Projekt
`scikit-learn`	Klassifikation mit Random Forest
`SciPy`	Gepaarter Wilcoxon-Test und Graphmerkmale
`statsmodels`	Holm-Korrektur und McNemar-Test
`pandas`	Datenhaltung und Aggregation
`NumPy`	numerische Grundoperationen
`matplotlib`	Visualisierung
Eigene Utilities	Graphmerkmale und Heuristiken

3.10. Evaluationsprotokoll

Die Auswertung basiert auf einer einheitlichen Versuchsplanung, damit die Ergebnisse der vier Heuristiken pro Instanz vergleichbar sind. Für jede Instanz werden alle Verfahren mit denselben vorgegebenen Laufzeitgrenzen ausgeführt und jeweils mit einer kleinen, festen Menge von Seeds gestartet. Alle übrigen Modellannahmen entsprechen den zuvor definierten Einstellungen zum TTDP und werden hier nicht erneut erläutert.

Jeder Lauf liefert genau einen Eintrag mit Instanzkennung, verwendetem Verfahren, vorgegebenem Zeitbudget, Seed, erzieltem Score, Tourlänge bzw. verbleibendem Budget und tatsächlich benötigter Laufzeit. Auf dieser Ebene können zunächst rein empirische Vergleiche zwischen den Verfahren vorgenommen werden. Für die spätere Algorithmenauswahl wird diese Laufebene auf Instanzebene verdichtet, indem je Instanz und Zeitbudget der beste beobachtete Score bestimmt und bei Gleichstand nach der kürzeren Tour ausgewählt wird. Dieses verdichtete Ziel dient anschließend als Label für das Lernverfahren.

4. Ergebnisse

Dieses Kapitel stellt die empirischen Ergebnisse der Algorithmenauswahl vor. Untersucht werden die vier fest definierten Heuristiken *greedy*, *grasp*, *ils* und *vns* sowie zwei Auswahlmodelle: ein globales Random-Forest-Modell über alle Zeitlimits und per-Limit Modelle je Zeitlimit. Das Versuchsdesign, die Bewertung nach Score mit Tie-Break über die Distanz, die Budgettoleranz ε, die Zeitlimits und die Referenzen SBS und VBS sind in den vorherigen Kapiteln festgelegt. Die Auswertung folgt den dort beschriebenen Regeln zur Aggregation und Trennung der Instanzen.

Die Darstellung folgt klaren Leitfragen. Zuerst wird gezeigt, wie sich der mittlere Score mit wachsender Laufzeit entwickelt und wie nah die Auswahl an die VBS-Leistung heranreicht. Dazu werden Plots des mittleren Scores, den Abstand zum VBS über die Limits, Within-Raten für $\delta \in \{0{,}1\%, 1\%, 5\%\}$ und Leistungsprofile der Score-zu-VBS-Quoten verwendet. Anschließend wird die Auswahl scorebasiert mit dem SBS auf Instanzebene verglichen und über Win, Tie und Loss sowie den Anteil nicht unterlegener Fälle berichtet. Für die Unsicherheit werden die mittlere Score-Differenz, 95 %-Konfidenzintervalle angegeben und die gepaarten Unterschiede im VBS-Abstand mit einem Wilcoxon-Test geprüft. Für die Klassifikationssicht werden Accuracy, macro-F1 und die Confusion-Matrizen gezeigt. Die Merkmalsbeiträge werden mithilfe von Permutationsauswertungen erläutert, sowohl global und klassenweise.

Alle Plots und Tabellen beziehen sich auf den Testsatz mit instanzweiser Trennung zum Training. Aggregationen über Seeds erfolgen als Mittelwerte je Instanz und Limit. Die nachfolgenden Abschnitte sind so aufgebaut, dass jede Fragestellung eine kurze Einordnung, die wesentlichen Befunde und eine knappe Interpretation enthält.

4.1. Erhöht die Auswahl den mittleren Score gegenüber SBS?

Zielsetzung

Dieser Abschnitt prüft, ob die Algorithmenauswahl den mittleren Score gegenüber einem festen Verfahren steigert und wie nah sie an die beobachtete Obergrenze herankommt. Grundlage sind pro Zeitlimit jeweils 700 Testinstanzen.

Darstellung

Die Abb. 1 zeigt auf der x-Achse die Zeitlimits von 1, 5, 10, 20 und 40 s. Auf der y-Achse steht der mittlere Score über alle Instanzen des jeweiligen Limits. Die obere Linie entspricht dem VBS als Referenz. Die untere Linie zeigt den SBS pro Limit. Dazwischen liegen die Kurven der Auswahlmodelle. Der vertikale Abstand einer Modellkurve zur VBS-Linie beschreibt die verbleibende Lücke zum beobachteten Optimum. Der Abstand zur SBS-Linie kennzeichnet den Gewinn durch die Auswahl gegenüber einem festen Verfahren. Die Steigung der Linien zeigt, wie gut zusätzliche Zeit in Score umgesetzt wird.

Ergebnisse

Alle Modelle liegen in jedem Limit über der SBS-Referenz und unter der VBS-Obergrenze. Bei 1 s erzielt *per-limit-1.0* einen mittleren Score von 567,2 Punkten und liegt damit um 0,39 Punkte über SBS. Die Lücke zu VBS beträgt 17,1 Punkte. Bei 5 s ist *per-limit-5.0* mit 700,5 Punkten vorn und gewinnt 5,88 Punkte gegenüber SBS. Der Abstand zu VBS sinkt hier auf 12,05 Punkte. Ab 10 s ist das *global* Modell am stärksten. Es erreicht 755,6 Punkte bei 10 s, 828,1 Punkte bei 20 s und 920,8 Punkte bei 40 s. Die Zugewinne gegenüber SBS steigen auf 8,41, 13,51 und 18,41 Punkte. Die Abstände zu VBS bleiben in diesem Bereich stabil klein und liegen bei 14,17, 14,69 und 14,66 Punkten. In relativen Quoten entspricht das ab 5 s mindestens 98 % des VBS-Scores.

Interpretation

Die Auswahl nutzt zusätzliche Rechenzeit wirkungsvoll. Mit wachsendem Budget steigen die mittleren Scores deutlich, während die Lücke zu VBS klein bleibt. Bei sehr kurzen Budgets profitieren die per-Limit Varianten, weil sie die spezifische Dynamik der knappen Zeit besser abbilden. Ab 10 s setzt sich das globale Modell durch und wandelt die zusätzliche Zeit zuverlässig in Score um. Die Auswahl schließt einen wachsenden Anteil der Differenz zwischen SBS und VBS: etwa 2,2 % bei 1 s, 32,8 % bei 5 s, 37,2 % bei 10 s, 47,9 % bei 20 s und 55,7 % bei 40 s. Damit liefert sie bereits ab mittleren Limits einen klaren Mehrwert und arbeitet nahe an der beobachteten Obergrenze.

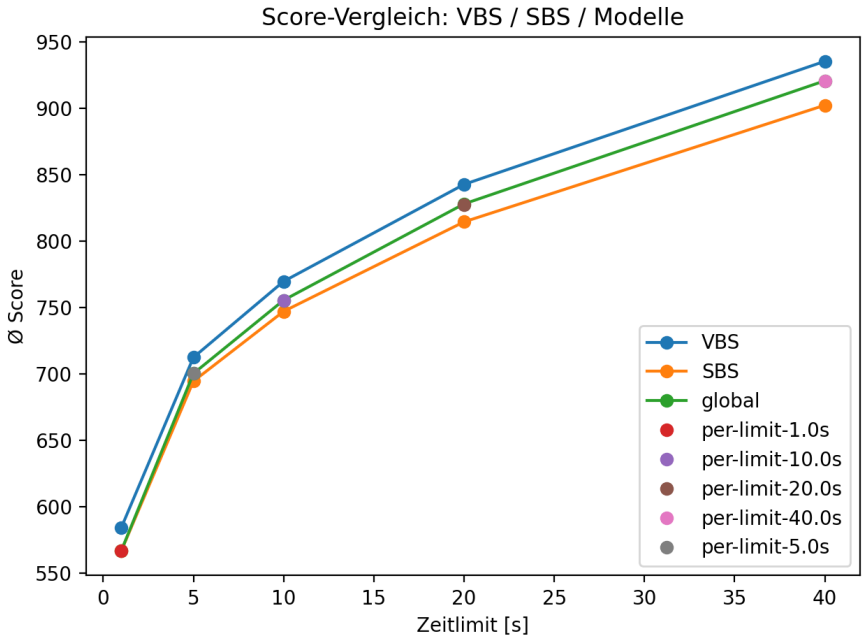


Abbildung 1: Mittlerer Score im Vergleich zu SBS und VBS über die Zeitlimits

4.2. Wie groß ist der Abstand zur VBS-Schranke je Limit?

Zielsetzung

Dieser Abschnitt untersucht, wie groß der verbleibende Abstand der Auswahlmodelle zur beobachteten Obergrenze ist. Der Abstand ist hier als Differenz zwischen VBS und mittlerem Score definiert. Die Auswertung basiert auf 700 Testinstanzen pro Zeitlimit.

Darstellung

Abb. 2 zeigt auf der x-Achse die Zeitlimits von 1, 5, 10, 20 und 40 s. Auf der y-Achse steht der mittlere Abstand, also VBS - mittlerem Score. Die durchgezogene Linie gehört zum globalen Modell. Die Punkte markieren die per-Limit Modelle bei ihrem jeweiligen Limit. Die gestrichelte Linie zeigt zum Vergleich den Abstand von SBS.

Ergebnisse

Das globale Modell weist bei 1 Sekunde einen Abstand von 17,1 Punkten auf und sinkt bei 5 Sekunden auf 12,1 Punkte. Ab 10 s bleibt der Abstand in einem engen Korridor mit 14,2 Punkten bei 10 Sekunden, 14,7 Punkten bei 20 s und 14,7 Punkten bei 40 s. Die per-Limit Modelle liegen dicht an dieser Linie und weichen in der Regel um etwa ein bis zwei Punkte ab. Der SBS entfernt sich mit steigender Zeit von der Obergrenze. Sein Abstand beträgt 17,5 Punkte bei 1 Sekunde, 17,9 Punkte bei 5 Sekunden, 22,6 Punkte bei 10 s, 28,2 Punkte bei 20 s und 33,1 Punkte bei 40 s.

Interpretation

Die Auswahl hält den Abstand zur VBS-Referenz über alle Limits klein. Besonders bei 5 s ist die Lücke am geringsten. Ab 10 s bleibt sie nahezu konstant und liegt bei rund vierzehn bis fünfzehn Punkten. Im Vergleich dazu entfernt sich SBS mit wachsendem Zeitbudget weiter von VBS. Die nahezu flache Linie des globalen Modells ab 10 s deutet auf stabile und robuste Entscheidungen hin. Die per-Limit Varianten liefern ein ähnliches Bild und unterscheiden sich nur geringfügig. In der Praxis bedeutet das, dass die Auswahl auch bei längeren Laufzeiten nahe an der beobachteten Obergrenze bleibt, während ein fixes Verfahren den Abstand nicht abbaut.

4.3. Wie oft liegt die Auswahl innerhalb $\delta \in \{0.1, 1, 5\}$ % des VBS?

Zielsetzung

Dieser Abschnitt misst, wie oft die Auswahlmodelle sehr dicht am beobachteten Optimum liegen. Das Kriterium *Within-*δ ist erfüllt, wenn der Modellscore mindestens $(1-\delta)$ so groß ist wie der VBS Score. Untersucht werden die Toleranzen $0,1\,\%$, $1\,\%$ und $5\,\%$. Die Auswertung basiert auf siebenhundert Testinstanzen pro Zeitlimit.

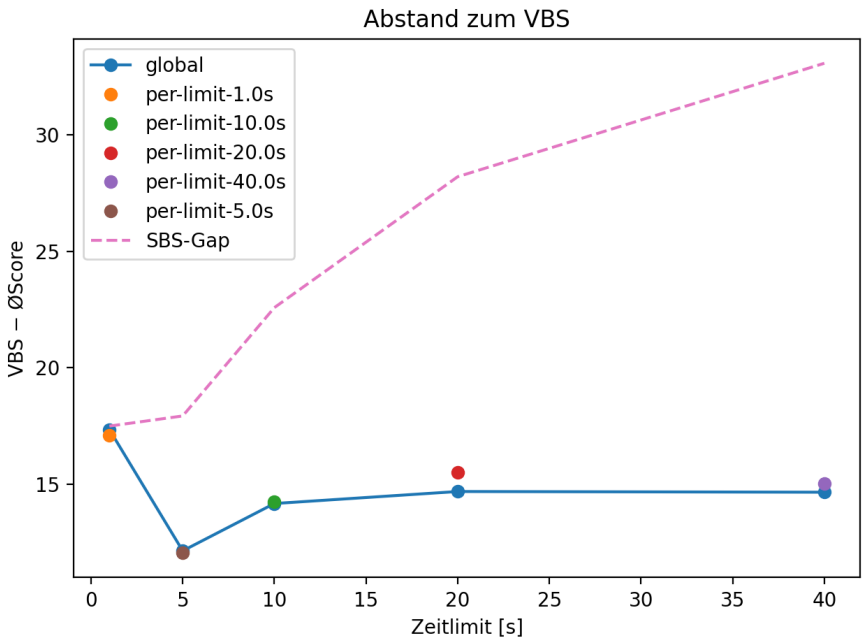


Abbildung 2: Abstand zum VBS der Modelle und des SBS über die Zeitlimits

Darstellung

Abb. 3 zeigt auf der x-Achse die Zeitlimits von 1, 5, 10, 20 und 40 Sekunden. Auf der y-Achse steht der Anteil der Instanzen, die das jeweilige *Within-δ* Kriterium erfüllen. Die durchgehende Linie gehört zum globalen Modell. Die Punkte markieren die per-Limit Modelle jeweils an ihrem eigenen Limit.

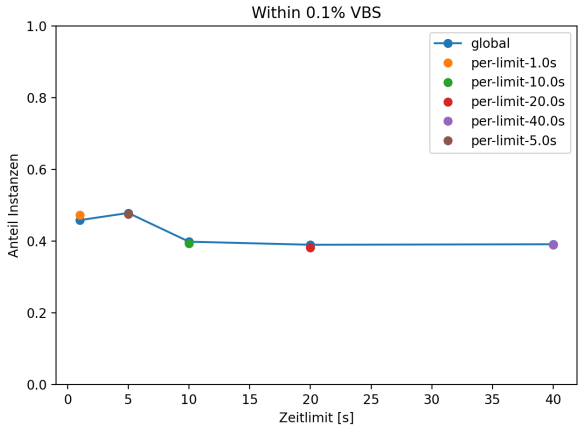
Ergebnisse

Bei $\delta = 0,1\,\%$ liegen je nach Limit rund 40 bis knapp 50 Prozent der Instanzen so dicht am VBS (siehe Abb. 3a). Das Maximum tritt bei 5 Sekunden auf und fällt bei längeren Limits leicht ab. Bei $\delta = 1\,\%$ liegt der Anteil mehrheitlich im Bereich von etwa 53 bis 56 Prozent und zeigt ein stabiles Niveau ab 10 Sekunden(siehe Abb. 3b). Bei $\delta = 5\,\%$ erreicht die Auswahl deutlich höhere Abdeckungen(siehe Abb. 3c). Je nach Limit liegen ungefähr 88 bis 90 Prozent der Instanzen innerhalb dieser Toleranz. Die per-Limit Modelle liegen nahe an der globalen Linie und weichen nur wenig ab.

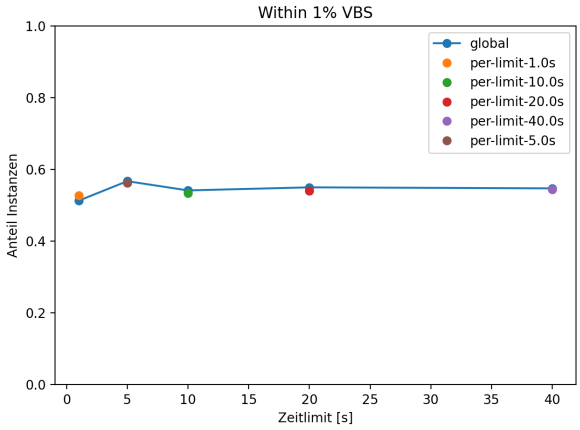
Interpretation

Die Auswahl erreicht schon bei sehr strenger Toleranz von 0,1 % etwa die Hälfte aller Instanzen. Mit einer Toleranz von 1 % ist die Mehrheit der Instanzen sehr nah am VBS. Bei 5 % ist die Annäherung an das Optimum fast flächendeckend. Die Kurven verlaufen ab 10 Sekunden weitgehend stabil. Die Beobachtung bestätigt, dass der Abstand zur Obergrenze klein bleibt

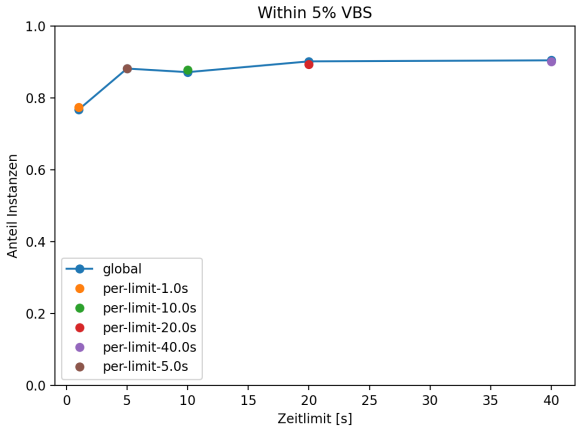
und sich mit zusätzlicher Zeit nicht mehr wesentlich verändert (siehe Abb. 2). Für die Praxis heißt das, dass die Auswahl bereits bei mittleren Budgets auf einem großen Teil der Instanzen nahezu das beobachtete Optimum erreicht.



(a) $\delta = 0{,}1\,\%$



(b) $\delta = 1\,\%$



(c) $\delta = 5\,\%$

Abbildung 3: Anteil der Instanzen innerhalb eines δ des VBS Scores über die Zeitlimits

4.4. Welche systematischen Verwechslungen zwischen {greedy, grasp, ils, vns} treten auf?

Zielsetzung

Es wird untersucht, welche Heuristiken vom Modell verwechselt werden und wie zuverlässig die Zuordnung pro Zeitlimit gelingt. Die Accuracy dient als kompakte Übersicht. Die vollständigen Matrizen zeigen die Fehlerstruktur.

	pred_grasp	pred_greedy	pred_ils	pred_vns
true_grasp	45	6	3	4
true_greedy	20	16	3	14
true_ils	16	11	21	7
true_vns	118	67	14	335

Tabelle 2: Confusion-Matrix des globalen Random-Forest-Modells für das Zeitlimit von 1 s (Zeilen: wahre Klasse, Spalten: vorhergesagte Klasse).

Tabelle 3: Accuracy in Prozent je Zeitlimit. Globales Modell und per-Limit Variante

	1 s	5 s	10 s	20 s	40 s
global	59,6	44,9	46,0	50,7	52,7
per-Limit	59,6	45,7	46,7	50,1	52,6

Darstellung

Eine Confusion Matrix ordnet die wahren Klassen zeilenweise und die Vorhersagen spaltenweise an (siehe beispielhaft Tab. 2). Hohe Werte auf der Diagonale stehen für korrekte Zuordnungen. Off-Diagonalen zeigen Verwechslungen. Alle Matrizen sind dem Anhang beigefügt (siehe Abschnitt A). Im Haupttext wird die Accuracy je Limit dargestellt und die häufigsten Verwechslungen beschrieben.

Ergebnisse

Die Accuracy (Tab. 3) des globalen Modells beträgt 59,6 % bei 1 s, 44,9 % bei 5 s, 46,0 % bei 10 s, 50,7 % bei 20 s und 52,7 % bei 40 s. Die per-Limit Varianten liegen sehr nah an diesen Werten mit 59,6 %, 45,7 %, 46,7 %, 50,1 % und 52,6 %. Die größte Verwechslung betrifft vns nach grasp und nimmt mit der Zeit zu.

Interpretation

Die häufige Verwechslung zwischen vns und grasp ist plausibel, da beide Verfahren bei mehr Zeit ähnlich stark werden. Fehler betreffen damit überwiegend zwei leistungsfähige Verfahren und kosten nur wenige Punkte. Das erklärt die kleine Lücke zum VBS in den Score-Analysen. Die geringeren Trefferraten bei greedy und ils folgen aus deren geringerer Basisrate im Testset und deuten nicht auf eine Fehlsteuerung hin.

4.5. Dominiert die Auswahl die SBS-ECDF der Score/VBS-Quoten?

Zielsetzung

Dieser Abschnitt bewertet die relative Güte der Lösungen über das ganze Testset. Dazu wird für jede Instanz die Quote aus erreichtem Score und der beobachteten Obergrenze VBS betrachtet. Das Leistungsprofile zeigt, wie groß der Anteil der Instanzen ist, die mindestens eine gegebene Quote erreichen. Verglichen werden das globale Modell und die per-Limit Variante.

Darstellung

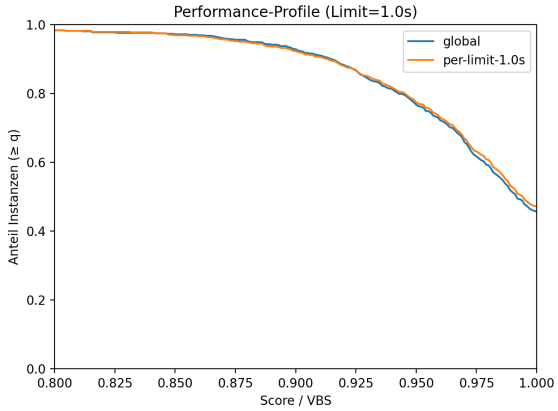
Abb. 4 zeigt auf der x-Achse die Quote Score/VBS im Bereich von 0,80 bis 1,00. Auf der y-Achse steht der Anteil der Instanzen, die diesen Wert mindestens erreichen. Eine Kurve ist umso besser, je weiter links und je weiter oben sie verläuft. Die vertikale Differenz an einer festen Position x zeigt, auf wie vielen Instanzen eine Methode diese Quote mindestens erreicht. Die VBS Grenze liegt bei einer Quote von eins.

Ergebnisse

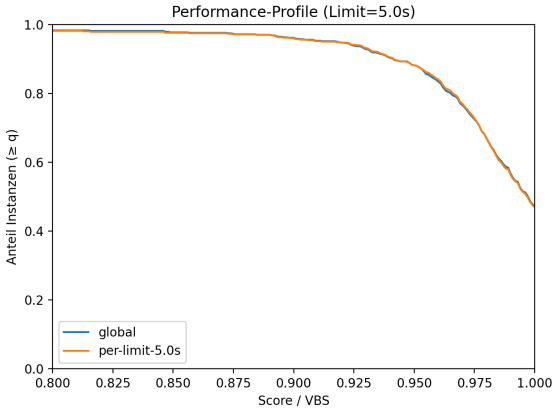
Die Kurven von global und per-Limit liegen in allen 5 Zeitbudgets sehr dicht beieinander. Bei 1 Sekunde zeigt die per-Limit Variante im rechten Bereich nahe 1,00 einen leicht höheren Anteil, während das globale Modell im mittleren Bereich kaum darunter liegt. Bei 5 Sekunden verlaufen beide Kurven fast deckungsgleich. Bei 10 und 20 Sekunden bleibt das Bild stabil, mit minimalen Vorteilen des globalen Modells im Bereich knapp unter 0,98. Bei 40 Sekunden ist der Verlauf weiterhin nahezu identisch, das globale Modell behält am rechten Rand einen kleinen Vorsprung. Insgesamt verschiebt sich die Masse mit wachsender Zeit deutlich in Richtung hoher Quoten, die Profile steigen im Bereich von 0,95 bis 1,00 steil an.

Interpretation

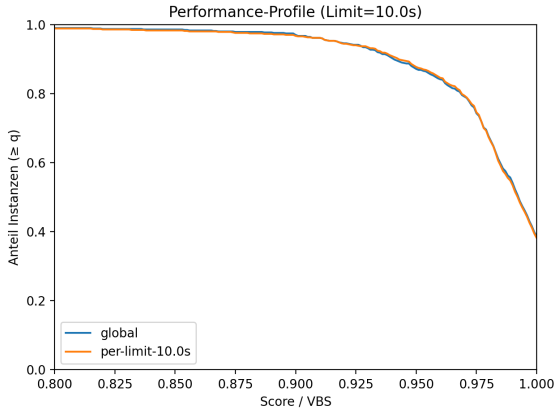
Die Auswahl liefert über weite Teile des Testsets eine hohe Quote relativ zum VBS. Die sehr kleinen Abstände zwischen globalem und per-Limit Modell bestätigen, dass beide Varianten konsistent entscheiden. Mit mehr Zeit wächst der Anteil der Instanzen nahe an eins deutlich an. Das stimmt mit der geringen VBS Lücke und den hohen *Within-δ* Raten aus den vorangehenden Auswertungen überein. Für die Praxis bedeutet es, dass die Auswahl nicht nur im Mittel zulegt, sondern die Verbesserungen über einen großen Teil der Instanzen trägt.



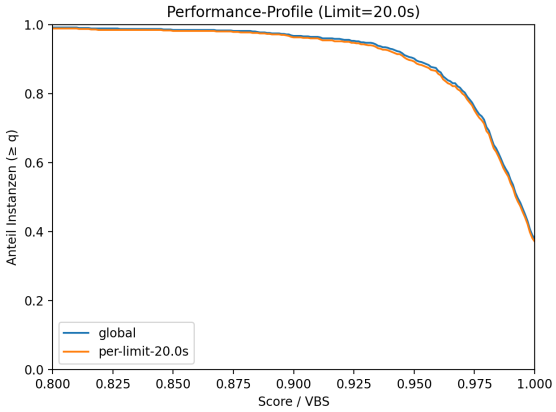
(a) 1 s



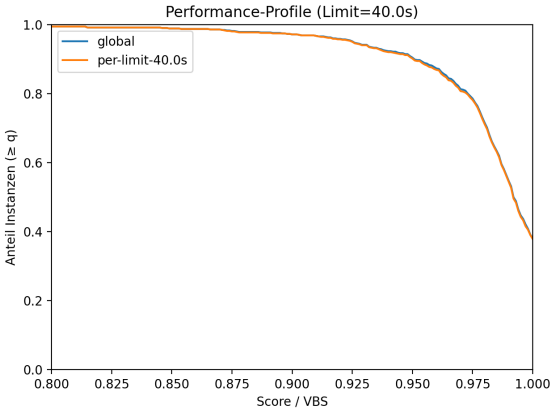
(b) 5 s



(c) 10 s



(d) 20 s



(e) 40 s

Abbildung 4: Leistungsprofile der Quote aus Score und VBS je Zeitlimit. Höher und weiter links bedeutet eine bessere Verteilung. Die Kurven von global und per-Limit liegen über allen Limits fast deckungsgleich.

4.6. Wie oft ist die Auswahl mindestens so gut wie der SBS?

Zielsetzung

Bewertet wird, wie oft die Auswahl im direkten Vergleich zum SBS beim Score gewinnt, gleichzieht oder verliert. Die Auswertung erfolgt pro Zeitlimit auf Instanzebene.

Darstellung

Die Tab. 4 und 5 enthalten vier Spalten. Ein *Win* bedeutet, dass der Score der Auswahl höher ist als der Score des SBS. Ein *Tie* entspricht einem exakt gleichen Score nach dem definierten Tie-Break. Ein *Loss* bedeutet einen niedrigeren Score als der SBS. Berichtet werden die Anteile je Limit sowie $p_{\geq \mathrm{SBS}} = \mathrm{Win} + \mathrm{Tie}$ für die Überlegenheit gegenüber dem SBS, getrennt für das globale Modell und die per-Limit Variante.

Ergebnisse

Die Anteile verschieben sich mit wachsender Laufzeit klar in Richtung *Win*. Beim globalen Modell steigt der Win-Anteil von 12,4 % (1 s) auf 47,6 % (40 s), während *Tie* von 79,4 % auf 48,0 % sinkt. *Loss* bleibt insgesamt niedrig und fällt am langen Zeitlimit auf 4,4 %. Damit wächst die Quote nicht unterlegener Fälle $p_{\geq \mathrm{SBS}}$ von 91,9 % auf 95,6 %. Die per–Limit Variante zeigt dasselbe Muster von mehr *Win*, weniger *Tie* und durchgehend geringe *Loss*–Anteile. Auffällig ist bei 1 s eine etwas höherer $p_{\geq \mathrm{SBS}}$ Wert als beim globalen Modell, weil dort *Loss* seltener ist.

Tabelle 4: scorebasiertes Win–Tie–Loss gegen den SBS (globales Modell), Anteile in %.

Limit	Win [%]	Tie [%]	Loss [%]	$p_{\geq \mathrm{SBS}}$ [%]
1 s	12,4	79,4	8,1	91,9
5 s	29,4	61,7	8,9	91,1
10 s	36,6	52,4	11,0	89,0
20 s	42,0	51,3	6,7	93,3
40 s	47,6	48,0	4,4	95,6

Tabelle 5: scorebasiertes Win–Tie–Loss gegen den SBS (per-Limit Modelle), Anteile in %.

Limit	Win [%]	Tie [%]	Loss [%]	$p_{\geq \mathrm{SBS}}$ [%]
1 s	11,3	82,0	6,7	93,3
5 s	29,9	61,0	9,1	90,9
10 s	37,4	51,4	11,1	88,9
20 s	41,7	50,4	7,9	92,1
40 s	47,0	47,9	5,1	94,9

Interpretation

Die Auswahl ist in der großen Mehrheit der Fälle mindestens so gut wie der SBS. Je nach Limit liegt $p_{\geq \mathrm{SBS}}$ beim globalen Modell zwischen 89,0 % und 95,6 %, bei der per-Limit Variante zwischen 88,9 % und 94,9 %. Mit mehr Zeit steigt der *Win*-Anteil und der *Loss*-Anteil sinkt. Die hohen *Tie*-Raten deuten auf Score-Plateaus hin und sind konsistent mit den Leistungsprofilen sowie den *Within-δ* Raten. Das erklärt, warum die Auswahl häufig vom SBS abweicht und dennoch im Mittel näher an der VBS-Leistung liegt.

4.7. Wie groß ist der Scoregewinn gegenüber dem SBS und sind die 95 % Intervalle positiv?

Zielsetzung

Der durchschnittliche Scorevorteil der Auswahl gegenüber dem SBS je Zeitlimit wird geschätzt, und die Unsicherheit wird als 95 %-Konfidenzintervalle angegeben.

Darstellung

Für jedes Zeitlimit L wird pro Instanz i die Differenz $\Delta_i(L) = \mathrm{Score}_i^{\mathrm{Modell}}(L) - \mathrm{Score}_i^{\mathrm{SBS}}(L)$ gebildet. Die beiden Scores stammen aus denselben Rohläufen und sind je Instanz und Limit als Mittelwerte über die Seeds aggregiert.

Aus den $\Delta_i(L)$ entsteht der Mittelwert $\bar{\Delta}(L)$ je Limit. Abb. 5 zeigt diesen Mittelwert als Punkt und das zugehörige 95 %-Konfidenzintervall als vertikale Strecke. Die Intervalle werden nichtparametrisch per Bootstrap über Instanzen berechnet: Für jedes Limit werden B = 5 000 Resamples der Instanzmenge mit Zurücklegen gezogen, in jedem Resample der Mittelwert der $\Delta_i(L)$ bestimmt und die 2,5 %- und 97,5 %-Quantile als Intervallgrenzen verwendet. Die Ziehungen sind durch einen festen Startwert reproduzierbar. Negative Werte bedeuten, dass die Auswahl im Mittel hinter dem SBS bleibt. Positive Werte zeigen einen durchschnittlichen Gewinn gegenüber dem SBS.

Ergebnisse

Für das globale Modell steigen die mittleren Gewinne mit dem Budget. Bei 1 s liegen sie bei $0,15$ Punkten, bei 5 s bei $5,79$ Punkten, bei 10 s bei $8,41$ Punkten, bei 20 s bei $13,51$ Punkten und bei 40 s bei $18,41$ Punkten ($n = 700$ je Limit). Die Intervalle sind breit und schließen die Null in allen Limits ein. Die per-Limit Varianten weisen auf das gleiche Muster. Die zugehörige Abbildung fasst Mittelwerte und Intervalle zusammen.

Interpretation

Die Auswahl gewinnt im Mittel zunehmend Score, je mehr Zeit zur Verfügung steht. Die Unsicherheit ist spürbar, weil die Differenzen über Instanzen stark streuen. Das passt zu den Leistungsprofilen und den Within-Raten, die zeigen, dass ein großer Teil der Instanzen sehr nah an VBS liegt, während auf einem kleineren Teil negative Differenzen auftreten. Im Mittel überwiegen die Gewinne klar.

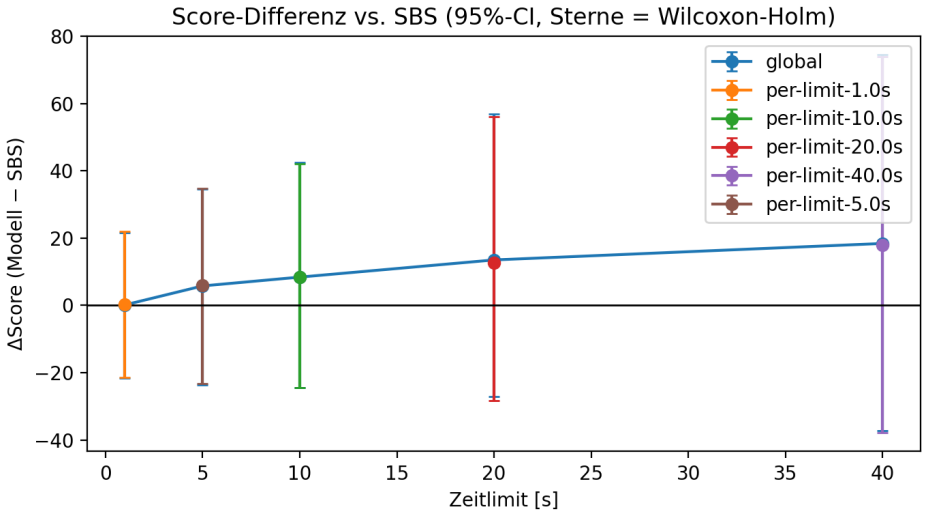


Abbildung 5: Score-Differenz gegenüber SBS mit 95 %-Konfidenzintervallen je Limit. Punkte sind Mittelwerte, Balken zeigen die Intervalle.

4.8. Verringert die Auswahl den Abstand gegenüber dem SBS signifikant und mit welchem Effekt?

Zielsetzung

Es wird geprüft, ob die Auswahl gegenüber SBS auf Instanzebene eine systematische Verbesserung im Abstand erzielt.

Darstellung

Pro Instanz und Limit wird die Abstand-Differenz $\Delta r = r_{\text{Modell}} - r_{\text{SBS}}$ betrachtet. Negative Werte bedeuten einen kleineren Abstand der Auswahl und sind damit besser. Der Wilcoxon Signed-Rank-Test wird gepaart über die Instanzen gerechnet. Berichtet werden Z-Statistik, holm-korrigierte p-Werte und der Effektgrößenparameter r.

Tabelle 6: Wilcoxon Signed Rank auf der Abstandsdifferenz Δr für das globale Modell

Limit	Z	Effektgröße $\lvert r \rvert$	p (Holm)
1 s	$-1,31$	0,049	1,000
5 s	$-2,09$	0,079	1,000
10 s	$-1,61$	0,061	1,000
20 s	$-2,35$	0,089	1,000
40 s	$-4,32$	0,163	1,000

Tabelle 7: Wilcoxon Signed Rank auf der Abstandsdifferenz Δr für die per-Limit-Modelle

Limit	Z	Effektgröße $\lvert r \rvert$	p (Holm)
1 s	$-1{,}28$	0,048	1,000
5 s	$-2{,}09$	0,079	1,000
10 s	$-1{,}61$	0,061	1,000
20 s	$-2{,}38$	0,090	1,000
40 s	$-4{,}35$	0,164	1,000

Ergebnisse

Für das globale Modell (vgl. Tab. 6) sind die Z-Werte in allen Limits negativ und nehmen mit der Zeit im Betrag zu ($Z = -1,31$ bei 1 s, $-2,09$ bei 5 s, $-1,61$ bei 10 s, $-2,35$ bei 20 s und $-4,32$ bei 40 s). Die Effektgrößen liegen im kleinen Bereich und wachsen von $|r| = 0,049$ auf $|r| = 0,163$. Die in der Datei ausgewiesenen Holm-korrigierten p-Werte sind nicht signifikant. Die Richtung der Z-Werte zeigt jedoch durchgängig auf *Modell* $<$ *SBS im Abstand* und ist damit konsistent zu den Hauptmetriken. Die per-Limit Modelle zeigen dasselbe Muster mit sehr ähnlichen Z-Werten und Effektgrößen (vgl. Tab. 7).

Interpretation

Die gepaarten Tests stützen die Tendenz der Mittelwerte. Die Auswahl verringert den Abstand relativ zu SBS auf vielen Instanzen, die Effekte liegen im kleinen Bereich und werden mit längerer Laufzeit stärker. Die fehlende formale Signifikanz in den gemeldeten p-Werten ist mit der breiten Streuung der Differenzen und der konservativen Korrektur erklärbar. Die Aussage deckt sich mit der kleinen VBS-Lücke sowie den hohen Within-Quoten.

Tabelle 8: Klassifikationsgüte je Limit: per-Limit vs. global

	1 s		5 s		10 s	
	Acc	$F1_{macro}$	Acc	$F1_{macro}$	Acc	$F1_{macro}$
per-Limit	0,8429	0,7572	0,8014	0,7389	0,8043	0,6834
global	0,8271 / 0,7138		0,8271 / 0,7138		0,8271 / 0,7138	
	20 s		40 s		Gesamt	
	Acc	$F1_{macro}$	Acc	$F1_{macro}$	Acc	$F1_{macro}$
per-Limit	0,8457	0,6636	0,8814	0,7158	—	—
global	0,8271 / 0,7138		0,8271 / 0,7138		0,8271	0,7138

4.9. Wie gut trifft die Auswahl je Zeitlimit die Algorithmenwahl pro Instanz und welche Verwechslungen prägen die Fehler?

Zielsetzung

Ergänzend zur Scoreanalyse wird geprüft, wie gut die Modelle je Zeitlimit zwischen den Heuristiken unterscheiden. Die per-Limit trainierten Modelle werden mit dem *globalen* Modell anhand von Accuracy und macro-F1 verglichen und die Ergebnisse in Bezug auf die Nähe zur VBS-Leistung eingeordnet.

Darstellung

Für jedes Zeitlimit werden Accuracy und macro-F1 auf dem Testsatz mit $n = 700$ Instanzen pro Limit berichtet und in Tab. 8 dargestellt. Das globale Modell wird auf dem gesamten Testsatz mit $n = 3500$ Instanzen ausgewertet. Die Kennzahlen stammen aus den zusammengefassten Ausgaben der Evaluation.

Ergebnisse

Die per-Limit trainierten Modelle erreichen je nach Budget hohe Accuracy und solide macro-F1-Werte: 1 s: Acc $= 0,8429$, $F1_{macro} = 0,7572$; 5 s: $0,8014$, $0,7389$; 10 s: $0,8043$, $0,6834$; 20 s: $0,8457$, $0,6636$; 40 s: $0,8814$, $0,7158$. Das globale Modell erzielt über alle Limits Acc $= 0,8271$ und $F1_{macro} = 0,7138$.

Interpretation

Die Klassifikationsgüte stützt das Muster der Scoreanalysen. Bei sehr knappen Budgets ist per-Limit leicht vorn. Mit wachsender Laufzeit nähern sich die Gütemaße an und das globale

Modell liefert über alle Limits hinweg stabile Werte. Das passt zur geringen Lücke der VBS-Leistung. Accuracy wird durch die Klassenverteilung geprägt. macro-F1 gleicht dies aus und zeigt, dass die seltenen Klassen nicht übersehen werden. Fehlentscheidungen zwischen grasp und vns sind häufig, kosten im Score aber meist wenig.

4.10. Wie zuverlässig ist die Auswahl je Zeitlimit und welche Merkmale tragen die Entscheidungen?

Zielsetzung

Die Entscheidungsqualität der Modelle wird bewertet und die Beiträge der Eingangsmerkmale werden erläutert. Die Gütemaße zeigen, wie zuverlässig die Auswahl die richtige Heuristik trifft. Die Merkmalsbeiträge geben an, welche Signale diese Entscheidungen tragen.

Darstellung

Die Gütemaße (Accuracy und macro-F1) werden je Zeitlimit auf dem Testsatz mit $n = 700$ Instanzen berechnet. Das globale Modell wird zusätzlich auf dem Gesamttest mit $n = 3500$ Instanzen berichtet. Accuracy misst den Anteil korrekt getroffener Entscheidungen. macro-F1 mittelt den F1-Wert gleich über die vier Klassen und gleicht damit die Klassenverteilung aus.

Zur Erklärung werden einzelne Merkmale über die Instanzen permutiert und der Güteabfall $\Delta M = M_{\text{original}} - M_{\text{permutiert}}$ gemessen. Global wird die Accuracy, klassenweise der F1 der jeweiligen Klasse verwendet. Großer Abfall bedeutet hohen Beitrag des Merkmals. Bei korrelierten Merkmalen können Beiträge auf Ersatzmerkmale ausweichen.

Ergebnisse

Die Gütemaße sind über alle Budgets stabil. Bei sehr knappen Budgets liegen die per-Limit Modelle leicht vorn. Ab 10 Sekunden liefert das globale Modell über alle Limits hinweg sehr konstante Werte. Die Permutation-Importance zeigt wiederkehrende Kernsignale. Beim globalen Modell ist `limit_s` das stärkste Merkmal. Es folgen budgetnahe Größen wie `budget` und `budget_per_node` sowie einfache Strukturmerkmale wie `nodes` und `mst__degree_avg`. Bei per-Limit Modellen dominieren bei kurzen Budgets `budget`, `budget_per_node` und einfache Strukturkennzahlen. Mit mehr Zeit bleiben Budgetsignale wichtig. MST-Kennzahlen wie Kantenausreißer und mittlere Grade gewinnen leicht an Einfluss.

Klassenweise Beiträge

Die jeweils zwei wichtigsten Merkmale pro Klasse im globalen Modell sind in Tab. 9 dargestellt. Die klassenweisen Profile sind konsistent mit den beobachteten Verwechslungen und erklären, warum viele Fehler wenig Score kosten. grasp und vns zeigen ein deutliches Zeitsignal über `limit_s` und reagieren zusätzlich auf einfache Strukturgrößen (`nodes`, `mst__degree_avg`). ils

Tabelle 9: Top-2 Merkmale pro Klasse im globalen Modell (Permutation-Importance nach F1 der Klasse)

Klasse	Top 1	Top 2
greedy	limit_s (0,308)	nodes (0,073)
grasp	limit_s (0,121)	budget_per_node (0,027)
ils	budget (0,274)	budget_per_node (0,210)
vns	limit_s (0,257)	nodes (0,017)

ist stark budgetgetrieben über `budget` und `budget_per_node`. greedy wird vor allem durch einfache Strukturmerkmale geprägt. Die ausführlichen Top-2-Listen je Zeitlimit stehen im Anhang (Tab. 19 bis 23) und beispielhaft .

Interpretation

Gütemaße und Merkmalsbeiträge ergänzen sich. Die Gütemaße belegen, dass die Auswahl zuverlässig funktioniert. Die Beiträge erklären das Muster zwischen per-Limit und global. Bei sehr kurzer Zeit hilft die Spezialisierung der per-Limit Modelle. Ab 10 Sekunden nutzt das globale Modell die Information `limit_s` besser und wählt häufiger die starke Heuristik. Die wiederkehrenden Kernmerkmale `limit_s`, `budget`, `budget_per_node`, `nodes` und `mst__degree_avg` tragen den Großteil der Entscheidung. Eine maßvolle Feature-Reduktion ist möglich, ohne die Güte sichtbar zu verschlechtern.

5. Diskussion und Einordnung der Ergebnisse

5.1. Zusammenfassung

Die Auswahl steigert den mittleren Score in allen Zeitlimits gegenüber dem SBS und rückt mit mehr Zeit näher an die VBS-Obergrenze. Der Zugewinn wächst beim globalen Modell von $+0,15$ Punkten bei 1 s bis $+18,41$ Punkten bei 40 s (vgl. Abb. 1). Der Abstand zum VBS bleibt klein und liegt je nach Limit etwa zwischen 12 und 17 Punkten (vgl. Abb. 2). Ab 5 s erreichen die Modelle rund 98 % der VBS-Leistung. Die Leistungsprofile verschieben sich mit der Laufzeit sichtbar nach links oben (vgl. Abb. 4).

5.2. Kernbefunde

5.2.1. Einordnung

Die Nähe zur Obergrenze zeigt sich doppelt: Der Abstand zum VBS bleibt gering und ein großer Teil der Instanzen liegt sehr nahe am VBS. Innerhalb von $0,1$ % des VBS liegen etwa 40 bis knapp 50 Prozent der Instanzen. Bei 1 % ist es gut die Hälfte, bei 5 % rund 90 % (vgl. Abb. 3a bis 3c). Das deutet darauf, dass typische Fehlentscheidungen nur wenige Punkte kosten.

5.2.2. Fehlerbild und Treffgenauigkeit

Die Confusion-Matrizen zeigen klare Muster. Die häufigste Verwechslung tritt zwischen vns und grasp auf, besonders bei längeren Budgets (siehe Abschnitt A). Beide Verfahren sind stark, daher sind die Scoreverluste durch diese Verwechslungen gering. Die Accuracy steigt mit der Laufzeit leicht an (vgl. Tab. 3).

5.2.3. Heterogenität und Risiko

Die instanzweisen Score-Differenzen streuen deutlich. Viele Fälle sind kleine Gewinne oder Ties, wenige Fälle weichen nach unten ab. Der Median der Differenzen liegt nahe bei null und der Mittelwert ist positiv. Das erklärt die breiten Intervalle, da häufiger kleine Verbesserungen und seltener deutliche Ausreißer nach oben oder unten reichen. In der Praxis sind die Verluste selten und klein. Die Auswahl ist in den meisten Fällen nicht schlechter als der SBS.

5.3. Scorebasierter Vergleich mit dem SBS

Die Auswertung vergleicht pro Instanz den Modellscore mit dem SBS-Score. Beim globalen Modell steigen die Win-Anteile mit der Zeit von etwa 12,4 % (1 s) auf 47,6 % (40 s). Die Tie-Anteile sinken von rund 79,4 % auf 48,0 %. Die Loss-Anteile bleiben niedrig und bewegen sich je nach Limit zwischen etwa 4 und 11 % (vgl. Tab. 4, 5). Damit wächst $p_{\geq \mathrm{SBS}}$ von rund 91,8 % auf 95,6 %. Die hohen Tie-Raten bei 1–5 s entstehen, weil die Auswahl häufig dieselbe Heuristik wie der SBS wählt und weil bei knappen Budgets viele Instanzen auf Score-Plateaus liegen, auf denen zusätzliche Verbesserungen kaum Punkte bringen.

5.4. Konfidenzintervalle und gepaarte Tests

Die 95 %-Konfidenzintervalle der mittleren Score-Differenz sind breit und schneiden in einzelnen Limits die Null (vgl. Abb. 5). Die Intervalle sind per Bootstrap über Instanzen mit $B = 5,000$ Ziehungen (Percentile-Methode, fester Startwert) bestimmt. Die Wilcoxon-Tests prüfen die pro Instanz berechneten Unterschiede im Abstand zum VBS. Die Effektgrößen sind klein und die nach Holm über die fünf Zeitlimits pro Modellvariante korrigierten p-Werte sind nicht signifikant (vgl. Tab. 6, 7). Die Richtung der Effekte passt zu allen Hauptmetriken, so dass sich mit mehr Zeit der Abstand zum VBS verringert.

5.5. Per-Limit gegenüber global

Bei sehr knappen Budgets liegt die per-Limit Variante vorn. Ab 10 Sekunden ist das globale Modell stärker und bleibt bis 40 s vorn. Die Leistungsprofile verlaufen ab 10 Sekunden nahezu deckungsgleich und zeigen am rechten Rand einen leichten Vorteil für das globale Modell (vgl. Abb. 4). Die Merkmalsbeiträge stützen diese Trennlinie. Das Feature `limit_s` ist im globalen Modell ein starkes Signal und erklärt den Vorsprung ab mittleren Budgets (vgl. Tab. 9, 21, 22, 23). Bei 1 bis 5s dominieren budgetnahe Größen und einfache Strukturmerkmale, was den Vorteil der per-Limit Modelle erklärt (vgl. Tab. 19, 20).

5.6. Alternative Erklärungen

Ein Teil der Ergebnisse hat einfache Ursachen. Erstens ist vns häufiger als die anderen Klassen. Häufige Klassen werden vom Modell öfter richtig erkannt und prägen die Accuracy. Zweitens enthalten mehrere Merkmale ähnliche Informationen. Wenn in der Permutationsauswertung ein Merkmal gestört wird, weicht das Modell auf ein anderes, ähnliches Merkmal aus. So wirken einzelne Beiträge klein, obwohl die Information insgesamt genutzt wird. Drittens nutzen die stochastischen Verfahren pro Instanz und Zeitlimit denselben Seed. Das macht die Vergleiche fairer und verringert die Streuung zwischen Wiederholungen. Diese Punkte ändern die Hauptaussagen nicht. Sie erklären jedoch warum die Accuracy nur moderat ausfällt, obwohl der Abstand zum VBS klein bleibt.

5.7. Praktische Implikationen

Ab mittleren Zeitbudgets bringt die Auswahl klaren Nutzen. Sie verkleinert die Lücke zum VBS deutlich und liegt oft nahe an der Obergrenze. Fehler kommen selten vor und kosten meist nur wenige Punkte. Bei sehr knappen Budgets ist die per-Limit Variante sinnvoll. Für den allgemeinen Einsatz ist das globale Modell die bessere Wahl.

5.8. Validität und Grenzen

Der VBS ist eine beobachtete Obergrenze der Experimente und kein garantiertes Optimum. Die Instanzen unterscheiden sich stark. Dadurch werden die Intervalle breit. Die Klassen sind ungleich verteilt. vns kommt häufiger vor. Train und Test sind strikt nach Instanz getrennt. Die stochastischen Verfahren nutzen pro Instanz und Zeitlimit dieselben Seeds. Das erhöht die Vergleichbarkeit und verringert die Streuung zwischen Wiederholungen. Wenn sich die Verteilung der Instanzen deutlich ändert, sollte die Auswertung erneut geprüft werden.

5.9. Schlussfolgerung

Die Auswahl ist über alle Zeitlimits robust und gewinnt mit mehr Zeit im Mittel hinzu. Der Abstand zur VBS-Leistung bleibt klein. Die typischen Fehler sind selten und kosten wenig Punkte. Bei sehr knappen Budgets ist die per-Limit Variante leicht im Vorteil. Ab 10 Sekunden überzeugt das globale Modell. Die Ergebnisse sind über die Metriken und Tests konsistent und stützen den praktischen Einsatz.

6. Ausblick

6.1. Ziel und Rahmen

Dieses Kapitel skizziert nächste Schritte, die über die hier gezeigten Experimente hinausgehen. Im Mittelpunkt stehen einfache Erweiterungen, die Nutzen, Robustheit und Verständlichkeit der Auswahl erhöhen.

6.2. Entscheidungen bei Unsicherheit

Die Auswahl soll nur dann entscheiden, wenn sie erkennbar sicher ist. Dazu wird eine einfache Schwelle auf der höchsten Modellwahrscheinlichkeit getestet. Liegt die Sicherheit darunter, greift die SBS-Referenz. Berichtet werden je Zeitlimit die Abdeckung der Modellentscheidungen, der mittlere Score und der Anteil nahe am VBS. So lässt sich eine sinnvolle Schwelle finden, die viel abdeckt und den Score hält.

6.3. Zweiter Versuch bei knappen Fällen

Viele Fehlentscheidungen betreffen grasp und vns. Ein zweiter Versuch nutzt dieses Muster. Das Zeitbudget wird auf zwei Läufe verteilt. Zuerst der Favorit und danach der Zweitplatzierte. Getestet werden feste Anteile wie 30/70, 50/50 und 70/30 sowie eine einfache Regel, die mehr Zeit gibt, wenn die beiden Wahrscheinlichkeiten nah beieinander liegen. Verglichen werden Score, Abstand zum VBS und der Anteil innerhalb von 1 und 5 % zum VBS.

6.4. Wahl zwischen global und per-Limit

Für sehr kurze Budgets war per-Limit leicht im Vorteil, ab 10 Sekunden das globale Modell. Eine kleine Entscheidungsregel kann diese Wahl automatisieren. Eingänge sind nur wenige Größen wie Zeitbudget und die höchste Modellwahrscheinlichkeit. Bewertet wird der Zugewinn beim Score und die Nähe zum VBS.

6.5. Robustheit auf anderen Instanzen

Die Auswahl wird auf veränderte Instanzmischungen geprüft. Dazu zählen andere Größen und Dichten, leicht veränderte Geometrien, andere Toleranzen und verrauschte Scores. Unabhängige Seeds erhöhen dabei die Aussagekraft. Ziel ist zu zeigen, dass die Befunde auch außerhalb des aktuellen Datensatzes tragen.

6.6. Aufwand und Latenz

Die Merkmale werden zu wenigen Gruppen gebündelt. Schrittweise werden Gruppen weggelassen, bis die Antwortzeit passt und der Score stabil bleibt. Einfache Strukturgrößen können vorab berechnet und wiederverwendet werden. So entsteht eine schlanke Konfiguration für den Betrieb.

6.7. Betrieb und Monitoring

Für den Einsatz werden pro Instanz die gewählte Heuristik, die höchste Wahrscheinlichkeit, Laufzeit und Score protokolliert. Ein regelmäßiger Vergleich auf einer Stichprobe gegen den SBS prüft die Stabilität. Bei deutlichen Verschiebungen der Instanzen wird das Modell mit aktuellen Daten neu trainiert. Die Trennung von Training und Test bleibt instanzweise erhalten.

7. Fazit

Diese Arbeit untersucht eine TTDP-Rundtour mit Budgetrestriktion auf vollständigen, metrischen Graphen und vergleicht vier feste Heuristiken greedy, grasp, ils und vns. Darauf aufbauend wurde ein Random Forest zur Algorithmenauswahl entwickelt und auf einem instanzweise getrennten Test ausgewertet. Die Bewertung folgt dem Score. Bei Gleichstand entscheidet die Distanz. Identische Seeds pro Instanz und Zeitlimit sichern die Vergleichbarkeit der stochastischen Läufe. So entsteht ein konsistentes Protokoll für faire Vergleiche.

Die Auswahl verbessert den mittleren Score in allen Zeitlimits gegenüber dem SBS und rückt mit mehr Zeit näher an die VBS-Obergrenze. Beim globalen Modell steigt der Zugewinn von $+0,15$ Punkten bei 1 s bis $+18,41$ Punkten bei 40 s. Ab 5 s werden im Mittel etwa 98 % der VBS-Leistung erreicht. Die Abstände zum VBS bleiben klein. Die Leistungsprofile verschieben sich mit wachsender Zeit sichtbar nach links oben. Die *Within-δ*-Raten zeigen, dass viele Instanzen sehr nah an der VBS-Leistung liegen. Innerhalb von $0,1$ % des VBS liegt rund die Hälfte des Tests nahe an der Obergrenze. Bei 1 % ist es gut die Hälfte. Bei 5 % werden ungefähr 90 % erreicht. Die typischen Fehlentscheidungen kosten damit nur wenige Punkte.

Die Fehlerstruktur ist gut verständlich. Die häufigste Verwechslung entsteht zwischen vns und grasp, vor allem bei längeren Budgets. Beide Verfahren sind stark, daher bleiben die Scoreverluste bei einer Vertauschung gering. Die Accuracy steigt mit der Laufzeit leicht an und bleibt durch die ungleiche Klassenverteilung moderat. Die instanzweisen Score-Differenzen streuen. Viele Fälle sind kleine Gewinne oder Gleichstände. Negative Ausreißer sind selten und fallen klein aus. Der Anteil der Fälle, in denen die Auswahl mindestens so gut ist wie der SBS, wächst über die Limits hinweg auf etwa 96 %. Das stützt das Bild einer robusten und im Mittel klar vorteilhaften Auswahl.

Die statistischen Analysen ordnen diese Effekte ein. Die Konfidenzintervalle der mittleren Score-Differenz sind breit und schneiden in einzelnen Limits die Null. Die Richtung ist jedoch über alle Zeitbudgets konsistent und zeigt eine Annäherung an die VBS-Leistung. Die gepaarten Wilcoxon-Tests auf dem Abstand zum VBS ergeben kleine Effektgrößen. Nach Holm-Korrektur über die fünf Limits sind die p-Werte nicht signifikant. Die Breite der Intervalle erklärt sich durch die Heterogenität der Instanzen. Die Kernaussagen bleiben stabil.

Beide Modellvarianten haben ihren Platz. Bei sehr knappen Budgets ist die per-Limit Variante leicht im Vorteil. Ab zehn Sekunden überzeugt das globale Modell und bleibt bis vierzig Sekunden vorn. Die Merkmalsbeiträge erklären diese Trennlinie. Das Zeitmerkmal `limit_s` ist im globalen Modell sehr informativ und unterstützt die Auswahl bei mittleren und langen Laufzeiten. Bei 1 bis 5 Sekunden dominieren Budget und einfache Strukturgrößen wie Anzahl der Knoten und mittlere Grade. Diese Kerngruppe trägt den Großteil der Entscheidung und erklärt die Nähe zur VBS-Grenze.

Praktisch lohnt sich die Auswahl ab mittleren Zeitbudgets deutlich. Sie schließt einen großen Teil der Lücke zwischen fester Heuristik und VBS. Die typischen Fehler sind selten und kosten wenig Punkte. In Szenarien mit sehr strengen Budgets empfiehlt sich die per-Limit Variante. Für die allgemeine Nutzung ist das globale Modell die sichere Wahl. Für den Einsatz genügen wenige stabile Merkmale. Zeitbudget, Budget pro Knoten, Gesamtbudget, Anzahl der Knoten und einfache Gradmaße liefern den größten Beitrag. Eine schlanke Konfiguration senkt die Vorbereitungszeit und hält den Score stabil.

Die Arbeit hat Grenzen. Der VBS ist eine beobachtete Obergrenze und nicht zwingend das wahre Optimum. Die Effekte streuen über Instanzen. Die Klassen sind ungleich verteilt und begünstigen vns. Die Seeds der stochastischen Verfahren sind pro Instanz und Limit synchronisiert. Das erhöht die Vergleichbarkeit und verringert die Varianz über Wiederholungen. Bei deutlicher Verschiebung der Instanzverteilung sollte die Auswertung erneuert werden.

In Summe zeigt die Studie, dass eine datengetriebene Algorithmenauswahl auf TTDP-Instanzen zuverlässig funktioniert. Sie liefert im Mittel klare Gewinne gegenüber dem SBS, bleibt nah an der VBS-Leistung und ist in ihren Fehlern mehrheitlich unkritisch. Die Ergebnisse sind über Metriken und Tests konsistent. Damit liegt eine solide Grundlage für den praktischen Einsatz und für gezielte Erweiterungen in künftigen Arbeiten vor.

Literaturverzeichnis

[1] G. Biau und E. Scornet, „A random forest guided tour“, *TEST*, Jg. 25, Nr. 2, S. 197–227, Juni 2016. DOI: 10.1007/s11749-016-0481-7.

[2] B. Bischl u. a., „ASlib: A Benchmark Library for Algorithm Selection“, *Artificial Intelligence*, Jg. 237, S. 41–58, 2016. DOI: 10.1016/j.artint.2016.04.003.

[3] L. Breiman, „Random Forests“, *Machine Learning*, Jg. 45, Nr. 1, S. 5–32, Okt. 2001. DOI: 10.1023/A:1010933404324.

[4] I.-M. Chao, B. L. Golden und E. A. Wasil, „The team orienteering problem“, *European Journal of Operational Research*, Jg. 88, Nr. 3, S. 464–474, 1996. DOI: 10.1016/0377-2217(94)00289-4.

[5] G. A. Croes, „A Method for Solving Traveling-Salesman Problems“, *Operations Research*, Jg. 6, Nr. 6, S. 791–812, Dez. 1958. DOI: 10.1287/opre.6.6.791.

[6] M. Dell'Amico, F. Maffioli und P. Värbrand, „On Prize-collecting Tours and the Asymmetric Travelling Salesman Problem“, *International Transactions in Operational Research*, Jg. 2, Nr. 3, S. 297–308, 1995. DOI: 10.1111/j.1475-3995.1995.tb00023.x.

[7] R. Diestel, *Graph Theory* (Graduate Texts in Mathematics), 6. Aufl. Berlin, Heidelberg: Springer, 2025, Bd. 173, Hardcover: 13 Dec 2024; eBook: 20 Jan 2025. DOI: 10.1007/978-3-662-70107-2.

[8] B. Efron und R. J. Tibshirani, *An Introduction to the Bootstrap*. Mai 1994. DOI: 10.1201/9780429246593.

[9] D. Feillet, P. Dejax und M. Gendreau, „Traveling Salesman Problems with Profits“, *Transportation Science*, Jg. 39, Nr. 2, S. 188–205, 2005. DOI: 10.1287/trsc.1030.0079.

[10] A. Fisher, C. Rudin und F. Dominici, „All Models are Wrong, but Many are Useful: Learning a Variable's Importance by Studying an Entire Class of Prediction Models Simultaneously“, *Journal of machine learning research : JMLR*, Jg. 20, Jan. 2019.

[11] D. Gavalas, C. Konstantopoulos, K. Mastakas und G. Pantziou, „A survey on algorithmic approaches for solving tourist trip design problems“, *Journal of Heuristics*, Jg. 20, Nr. 3, S. 291–328, 2014. DOI: 10.1007/s10732-014-9242-5.

[12] B. L. Golden, L. Levy und R. Vohra, „The orienteering problem“, *Naval Research Logistics (NRL)*, Jg. 34, Nr. 3, S. 307–318, 1987. DOI: https://doi.org/10.1002/1520-6750(198706)34:3<307::AID-NAV3220340302>3.0.CO;2-D.

[13] A. Gunawan, H. C. Lau und K. Lu, „An Iterated Local Search Algorithm for Solving the Orienteering Problem with Time Windows“, in *Evolutionary Computation in Combinatorial Optimization (EvoCOP 2015)*, Ser. LNCS, Bd. 9026, Cham: Springer, 2015, S. 61–73. DOI: 10.1007/978-3-319-16468-7_6.

[14] G. Gutin und A. P. Punnen, Hrsg., *The Traveling Salesman Problem and Its Variations* (Combinatorial Optimization). Dordrecht: Kluwer Academic Publishers, 2002, Bd. 12. DOI: 10.1007/b101971.

[15] S. Holm, „A Simple Sequentially Rejective Multiple Test Procedure“, *Scandinavian Journal of Statistics*, Jg. 6, Nr. 2, S. 65–70, 1979. Adresse: http://www.jstor.org/stable/4615733.

[16] F. Hutter, L. Xu, H. H. Hoos und K. Leyton-Brown, „Algorithm Runtime Prediction: Methods & Evaluation“, *Artificial Intelligence*, Jg. 206, S. 79–111, 2014. DOI: 10.1016/j.artint.2013.10.003.

[17] M. Keshtkaran und K. Ziarati, „A novel GRASP solution approach for the Orienteering Problem“, *Journal of Heuristics*, Jg. 22, Nr. 5, S. 699–726, 2016. DOI: 10.1007/s10732-016-9316-7.

[18] L. Kotthoff, „Algorithm Selection for Combinatorial Search Problems: A Survey“, *AI Magazine*, Jg. 35, Nr. 3, S. 48–60, 2014. DOI: 10.1609/aimag.v35i3.2460.

[19] Q. McNemar, „Note on the sampling error of the difference between correlated proportions or percentages“, *Psychometrika*, Jg. 12, Nr. 2, S. 153–157, Juni 1947. DOI: 10.1007/BF02295996.

[20] M. E. J. Newman, *Networks*, 2. Aufl. Oxford: Oxford University Press, 2011.

[21] R. Pěnička, J. Faigl und M. Saska, „Variable Neighborhood Search for the Set Orienteering Problem and its application to other Orienteering Problem variants“, *European Journal of Operational Research*, Jg. 276, Nr. 3, S. 816–825, 2019. DOI: 10.1016/j.ejor.2019.01.047.

[22] J. R. Rice, „The Algorithm Selection Problem“, in *Advances in Computers*, M. Rubinoff und M. C. Yovits, Hrsg., Bd. 15, New York: Academic Press, 1976, S. 65–118. DOI: 10.1016/S0065-2458(08)60520-3.

[23] M. Sokolova und G. Lapalme, „A systematic analysis of performance measures for classification tasks“, *Information Processing & Management*, Jg. 45, S. 427–437, Juli 2009. DOI: 10.1016/j.ipm.2009.03.002.

[24] P. Toth und D. Vigo, *Vehicle Routing*, D. Vigo und P. Toth, Hrsg. Philadelphia, PA: Society for Industrial und Applied Mathematics, 2014. DOI: 10.1137/1.9781611973594.

[25] T. Tsiligirides, „Heuristic Methods Applied to Orienteering“, *Journal of the Operational Research Society*, Jg. 35, Nr. 9, S. 797–809, 1984. DOI: 10.1057/jors.1984.162.

[26] P. Vansteenwegen, W. Souffriau und D. V. Oudheusden, „The orienteering problem: A survey“, *European Journal of Operational Research*, Jg. 209, Nr. 1, S. 1–10, 2011. DOI: 10.1016/j.ejor.2010.03.045.

[27] D. J. Watts und S. H. Strogatz, „Collective dynamics of ‘small-world’ networks“, *Nature*, Jg. 393, Nr. 6684, S. 440–442, 1998. DOI: 10.1038/30918.

[28] D. B. West, *Introduction to Graph Theory*, 2. Aufl. Upper Saddle River, NJ: Prentice Hall, 2001.

[29] F. Wilcoxon, „Individual Comparisons by Ranking Methods“, in *Breakthroughs in Statistics: Methodology and Distribution*, S. Kotz und N. L. Johnson, Hrsg. New York, NY: Springer New York, 1992, S. 196–202. DOI: 10.1007/978-1-4612-4380-9_16.

[30] L. Xu, F. Hutter, H. H. Hoos und K. Leyton-Brown, „SATzilla: Portfolio-based Algorithm Selection for SAT“, *Journal of Artificial Intelligence Research*, Jg. 32, S. 565–606, 2008. DOI: 10.1613/jair.2490.

A. Confusion-Matrizen

A.1. Globales Modell

	pred_grasp	pred_greedy	pred_ils	pred_vns
true_grasp	124	11	7	5
true_greedy	23	20	3	39
true_ils	38	4	31	5
true_vns	187	54	10	139

Tabelle 10: global: limit 5 s

	pred_grasp	pred_greedy	pred_ils	pred_vns
true_grasp	186	9	9	3
true_greedy	24	18	2	6
true_ils	41	4	21	2
true_vns	244	22	12	97

Tabelle 11: global: limit 10 s

	pred_grasp	pred_greedy	pred_ils	pred_vns
true_grasp	234	8	6	0
true_greedy	19	10	0	4
true_ils	41	3	16	0
true_vns	241	14	9	95

Tabelle 12: global: limit 20 s

	pred_grasp	pred_greedy	pred_ils	pred_vns
true_grasp	255	4	9	2
true_greedy	8	8	2	3
true_ils	47	2	23	0
true_vns	236	8	10	83

Tabelle 13: global: limit 40 s

A.2. per-Limit Modell

	pred_grasp	pred_greedy	pred_ils	pred_vns
true_grasp	37	11	5	5
true_greedy	13	17	5	18
true_ils	10	15	23	7
true_vns	116	65	13	340

Tabelle 14: per-limit: limit 1 s

	pred_grasp	pred_greedy	pred_ils	pred_vns
true_grasp	123	11	10	3
true_greedy	29	22	3	31
true_ils	32	5	36	5
true_vns	185	57	9	139

Tabelle 15: per-limit: limit 5 s

	pred_grasp	pred_greedy	pred_ils	pred_vns
true_grasp	190	10	7	0
true_greedy	31	12	4	3
true_ils	34	4	27	3
true_vns	242	23	12	98

Tabelle 16: per-limit: limit 10 s

	pred_grasp	pred_greedy	pred_ils	pred_vns
true_grasp	234	8	6	0
true_greedy	21	7	0	5
true_ils	40	4	16	0
true_vns	244	12	9	94

Tabelle 17: per-limit: limit 20 s

	pred_grasp	pred_greedy	pred_ils	pred_vns
true_grasp	254	6	9	1
true_greedy	10	8	2	1
true_ils	47	2	23	0
true_vns	238	7	9	83

Tabelle 18: per-limit: limit 40 s

B. Top-2 Merkmale

Tabelle 19: Top-2 Merkmale pro Klasse – 1 s

Klasse	Top 1	Top 2
greedy	mst___degree_avg (0,055)	nodes (0,047)
grasp	budget (0,095)	budget_per_node (0,073)
ils	budget (0,347)	rad_0_5___degree_avg (0,192)
vns	mst___degree_avg (0,013)	nodes (0,012)

Tabelle 20: Top-2 Merkmale pro Klasse – 5 s

Klasse	Top 1	Top 2
greedy	budget (0,072)	nodes (0,072)
grasp	budget_per_node (0,042)	budget (0,035)
ils	budget (0,310)	budget_per_node (0,207)
vns	budget (0,019)	budget_per_node (0,016)

Tabelle 21: Top-2 Merkmale pro Klasse – 10 s

Klasse	Top 1	Top 2
greedy	budget_per_node (0,162)	nodes (0,066)
grasp	budget_per_node (0,033)	budget (0,020)
ils	budget (0,245)	budget_per_node (0,226)
vns	nodes (0,030)	mst___degree_avg (0,019)

Tabelle 22: Top-2 Merkmale pro Klasse – 20 s

Klasse	Top 1	Top 2
greedy	budget_per_node (0,189)	budget (0,136)
grasp	budget_per_node (0,017)	budget (0,013)
ils	budget (0,222)	budget_per_node (0,169)
vns	nodes (0,006)	hull_vertices (0,005)

Tabelle 23: Top-2 Merkmale pro Klasse – 40 s

Klasse	Top 1	Top 2
greedy	nodes (0,129)	density (0,080)
grasp	budget_per_node (0,019)	budget (0,017)
ils	budget_per_node (0,283)	budget (0,263)
vns	nodes (0,009)	mst___degree_avg (0,007)

Selbstständigkeitserklärung

Hiermit versichere ich, dass ich die Bachelorarbeit selbstständig und nur unter Verwendung der angegebenen Quellen und Hilfsmittel angefertigt habe. Als Hilfsmittel kamen KI-gestützte Programmierassistenten zur Unterstützung beim Programmieren sowie Large-Language-Models zur Grammatik- und Rechtschreibprüfung zum Einsatz. Die aus fremden Quellen direkt oder indirekt übernommenen Stellen sind als solche kenntlich gemacht.

Die Arbeit wurde bisher in gleicher oder ähnlicher Form keiner anderen Prüfungsbehörde vorgelegt und auch nicht veröffentlicht.

Die digitalisierte Fassung der Arbeit stimmt mit der in Schriftform vorgelegten Fassung wörtlich überein.

Ort, Datum	Unterschrift